

WePlan

AI 周末生活规划师

“不只是推荐，而是帮你把事情做完”

美团 AI Hackathon 2026

赛题 06: 周末闲时活动规划

项目名称: WePlan – AI 周末生活规划师

参赛赛道: 赛题 06: 周末闲时活动规划

提交日期: 2026 年 5 月 16 日

版本: v1.0

WePlan – 七个 AI Agent 协作，一站式完成你的周末规划

参赛信息

赛事名称	美团 AI Hackathon 2026
参赛赛道	赛题 06: 周末闲时活动规划
项目名称	WePlan – AI 周末生活规划师
参赛者	戴上昊
学校	浙江大学
邮箱	daishanghao@zju.edu.cn
GitHub	https://github.com/anthropics/weplan
Demo 链接	https://weplan.demo.site
技术栈	MiniMax M2.7 + FastAPI + React + 高德 API
核心特性	7-Agent 协作、Critic 反馈循环、群体投票、Agent Thinking 可视化

项目摘要

WePlan 是一个基于多 Agent 协作的 AI 周末生活规划系统。系统采用 7 个专职 Agent(Orchestrator、Context、Dining、Activity、Plan Synthesizer、Critic、Execution/Notification) 协同工作, 通过 MiniMax M2.7 大模型驱动意图理解与方案生成, 集成高德地图 API 获取真实 POI 和天气数据, 实现从“用户说一句话”到“完整可执行周末方案”的全链路自动规划。

核心创新包括: Critic Agent 确定性校验反馈循环保证方案质量; 群体共识投票机制解决多人决策难题; Agent Thinking 可视化增强用户信任; 多方案 5 维雷达图对比辅助用户选择。系统支持 6 个预建 Demo 场景, 覆盖家庭出游、朋友聚会、情侣约会等典型周末场景。

目录

Part I: 项目概览10

1 赛题分析10

1.1 赛题背景10

1.2 赛题要求详解10

1.3 评审标准分析10

1.4 赛题挑战分析11

2 选题理由12

2.1 六道赛题对比分析12

2.2 选择赛题 06 的核心原因12

3 产品定位13

3.1 WePlan 不是什么13

3.2 WePlan 是什么13

3.3 目标用户画像14

4 核心创新点总结14

4.1 创新 1: 七 Agent 协作架构14

4.2 创新 2: Critic 确定性反馈循环14

4.3 创新 3: 群体共识投票机制15

4.4 创新 4: Agent Thinking 可视化15

4.5 创新 5: 多方案 5 维对比15

4.6 创新 6: Harness 工程层15

4.7 创新 7: 预建 Demo 场景15

5 Demo 场景预览15

Part II: 市场与竞品分析17

6 本地生活服务市场现状17

6.1 中国本地生活市场规模17

6.2 用户需求变化趋势17

6.3 AI 在本地生活领域的渗透18

4

7 美团 AI 产品生态分析	18
7.1 美团 “小团”AI 管家	18
7.2 美团 “小美”AI 生活秘书	19
7.3 美团 LongCat 大模型	19
7.4 美团三层 AI 架构	19
7.5 美团 AI 研发投入	20
8 竞品对比分析	20
8.1 Google Travel Canvas (2026)	20
8.2 MonkeyTravel	20
8.3 Meetup.AI (UC Berkeley)	21
8.4 携程 AI 助手	21
8.5 飞猪 AI 规划	21
8.6 综合对比表	22
9 用户痛点调研	22
9.1 痛点 1: 信息分散	22
9.2 痛点 2: 串联困难	23
9.3 痛点 3: 群体决策困难	23
Part III: 系统设计	24
10 整体架构设计	24
10.1 架构总览	24
10.2 设计哲学	24
10.3 架构对比	25
11 Agent 设计详解	26
11.1 Orchestrator Agent	26
11.2 Context Agent	28
11.3 Dining Agent	29
11.4 Activity Agent	31
11.5 Plan Synthesizer	32
11.6 Critic Agent	35
11.7 Execution Agent	37
11.8 Notification Agent	38

12 Harness Engineering 层	38
12.1 ToolHarness 类设计	39
12.2 异常处理矩阵	40
13 多层记忆系统	42
13.1 Session Memory	42
13.2 User Profile	43
13.3 Plan History	43
14 群体共识投票	45
14.1 投票流程设计	45
14.2 4 级投票机制	45
14.3 投票汇总与方案调整算法	45
15 数据流图	47
15.1 完整数据流	47
15.2 SSE 流式输出设计	48
Part IV: 技术实现	49
16 MiniMax M2.7 集成	49
16.1 API 调用方式	49
16.2 Function Calling 实现	50
16.3 流式输出处理	50
16.4 Token 管理与成本控制	51
17 高德地图 API 集成	52
17.1 Web Service API 概述	52
17.2 POI 搜索实现	53
17.3 路线规划实现	55
17.4 天气查询实现	56
17.5 API Key 管理与额度优化	57
17.6 缓存策略	57
18 后端实现	58
18.1 FastAPI 路由设计	58
18.2 SSE 流式响应实现	60
18.3 异步并发处理	61
18.4 API 端点清单	63

19 前端实现	63
19.1 双面板布局设计	63
19.2 聊天界面实现	64
19.3 时间线卡片渲染	64
19.4 雷达图绘制	66
19.5 Agent Thinking 面板	67
19.6 深色/浅色主题	67
19.7 响应式设计	68
20 部署架构	68
20.1 阿里云 ECS 配置	68
20.2 Nginx 反向代理	68
20.3 systemd 服务管理	69
20.4 性能优化	70
Part V: 数据设计	71
21 高德 API 真实数据	71
21.1 数据采集策略	71
21.2 缓存机制	71
21.3 额度管理	72
22 Mock 数据库	73
22.1 数据字段设计	73
22.2 杭州数据	74
22.3 北京数据	75
22.4 数据质量保证	75
23 预建案例设计	75
23.1 案例 D1: 杭州家庭亲子游	75
23.2 案例 D2: 北京朋友聚会	76
23.3 案例 D3–D6	76
Part VI: 测试与评估	78
24 功能测试	78
24.1 场景 A 测试: 家庭亲子游	78
24.2 场景 B 测试: 朋友聚会	78

- 24.3 场景 C 测试: 动态调整 79
- 25 性能指标 79
 - 25.1 响应时间 80
 - 25.2 方案准确率 80
 - 25.3 异常处理成功率 81
- 26 用户体验分析 81
 - 26.1 交互流畅度 81
 - 26.2 方案质量评估 81
 - 26.3 Agent Thinking 可读性 82
- 27 与竞品的定量对比 82
- Part VII: 商业价值分析 83
- 28 与美团小团产品的对接路径 83
 - 28.1 功能对齐分析 83
 - 28.2 技术接入方案 83
 - 28.3 数据协同策略 83
- 29 市场潜力 84
 - 29.1 本地生活 AI 服务市场规模预测 84
 - 29.2 目标用户画像 85
 - 29.3 用户增长潜力 85
- 30 商业模式 85
 - 30.1 免费基础 + 增值服务 86
 - 30.2 商家合作 (推荐佣金) 86
 - 30.3 数据价值 86
- 31 竞争优势 87
 - 31.1 全链路闭环 87
 - 31.2 群体决策差异化 87
 - 31.3 与美团生态的天然契合 87
- 32 发展路线图 87
 - 32.1 短期 (3 个月): 核心功能完善 87
 - 32.2 中期 (6 个月): 接入真实 API 87
 - 32.3 长期 (12 个月): 多场景扩展 88

33 未来技术演进	88
33.1 LLM 能力升级路径	88
33.2 Agent 架构演进	89
33.3 数据生态建设	89
33.4 用户体验持续优化	89
 Part VIII: 安全与隐私	 91
34 数据安全设计	91
34.1 数据分类	91
34.2 数据存储安全	91
34.3 数据传输安全	91
35 用户隐私保护	91
35.1 隐私设计原则	91
35.2 匿名化处理	92
36 API 安全	92
36.1 认证与授权	92
36.2 速率限制	93
36.3 输入验证	93
37 LLM 安全	93
37.1 Prompt Injection 防护	93
37.2 内容安全	94
37.3 安全审计与合规	94
 附录	 96
A API 接口完整清单	96
B 代码统计	97
C 参考文献	97
D 致谢	98

Part I: 项目概览

1 赛题分析

1.1 赛题背景

美团 AI Hackathon 2026 是美团面向全国高校开发者举办的人工智能编程马拉松，旨在挖掘和培养具有创新能力的 AI 技术人才。本次大赛共设置 6 道赛题，涵盖了从基础 AI 能力到复杂场景应用的多个维度。赛题 06“周末闲时活动规划”属于本地生活服务领域的智能规划类题目，要求参赛者构建一个能够理解用户需求、搜索本地服务信息、智能规划活动方案的 AI 系统。

赛题 06 的完整描述为：“设计一个智能活动规划 Agent，能够根据用户输入的时间、地点、人群偏好等信息，自动搜索附近餐厅、景点、娱乐设施，并生成一份完整的周末活动方案。方案需包含时间线、预算估算、交通建议，并支持动态调整。”

1.2 赛题要求详解

赛题 06 提出了以下具体要求：

- (1) **输入理解**：系统需理解自然语言描述的复杂需求，包括时间范围、地点偏好、人群构成（家庭/朋友/情侣）、预算约束、特殊需求（如无障碍设施、儿童适宜等）。
- (2) **信息搜索**：需要调用外部 API(如地图 API) 搜索真实的 POI(Point of Interest) 数据，包括餐厅、景点、娱乐设施等，并获取营业时间、评分、价格等详细信息。
- (3) **方案生成**：生成包含完整时间线的活动方案，需考虑各活动之间的距离和交通时间，确保方案在时间和空间上的可行性。
- (4) **预算管理**：估算每项活动的花费，提供总预算和分项预算，确保不超出用户设定的预算范围。
- (5) **动态调整**：支持根据天气变化、用户反馈、突发情况等动态调整方案。
- (6) **设计文档**：提供不超过 2 页的设计文档，说明 Planning 策略、工具调用链路、异常处理机制。

1.3 评审标准分析

根据赛事官方公布的评审标准，评委将从以下维度进行评分：

表 1: 评审标准权重分析

评审维度	预估权重	我们的应对策略
技术创新性	30%	7-Agent 架构 + Critic 反馈循环 + 群体投票
产品完成度	25%	完整前后端 + 6 个可交互 Demo 场景
用户体验	20%	Agent Thinking 可视化 + 双面板布局 + 深色主题
商业价值	15%	与美团小团对接路径 + 市场分析
代码质量	10%	模块化设计 + 异常处理 + 代码注释

1.4 赛题挑战分析

赛题 06 看似是一个简单的“搜索 + 推荐”问题，但深入分析后会发现其中蕴含多个技术挑战：

挑战 1: 多维约束优化

用户的需求本质上是一个多维约束满足问题 (CSP)。时间窗口、地理距离、预算限制、人群偏好、天气条件等约束之间存在复杂的交互关系。例如，选择了一个远离市中心的景点后，附近可选的餐厅数量会大幅减少，可能导致无法满足美食偏好的约束。这需要系统具备全局优化能力，而不是简单的贪心搜索。

挑战 2: 实时数据整合

系统需要整合来自多个数据源的实时信息: 天气数据 (影响户外活动可行性)、POI 营业状态 (周末可能有特殊营业时间)、交通状况 (影响活动之间的转场时间)。这些数据源的 API 调用存在延迟和失败的风险，需要设计健壮的数据获取策略。

挑战 3: 个性化与多样性平衡

生成的方案既要符合用户的个性化偏好，又要具备足够的多样性，让用户有选择空间。特别是在群体出行场景下，不同成员可能有截然不同的偏好 (如一人喜欢户外运动，另一人偏好室内文化活动)，系统需要找到各方都能接受的“帕累托最优”方案。

挑战 4: 方案可执行性

生成的方案不能是理论上的“完美计划”，而必须在实际执行层面可行。这意味着需要考虑：餐厅是否需要预订、景点是否需要提前购票、活动之间的步行/驾车时间是否合理、是否有足够的缓冲时间应对意外等。

2 选题理由

2.1 六道赛题对比分析

本次 Hackathon 共设 6 道赛题，我们对每道赛题进行了系统性的对比分析，最终选择了赛题 06：

表 2: 六道赛题对比分析					
赛题	主题	技术深度	创新空间	展示效果	与美团关联度
01	智能客服对话	中等	较小	一般	高 (客服场景)
02	文本摘要生成	中等	较小	一般	中 (内容理解)
03	代码辅助工具	高	中等	中等	低 (非核心业务)
04	多模态理解	很高	大	好	中 (商品识别)
05	知识图谱问答	高	中等	中等	中 (美食知识)
06	周末活动规划	高	很大	很好	很高 (本地生活)

2.2 选择赛题 06 的核心原因

原因一：与美团核心业务高度契合。美团的核心业务是本地生活服务，而周末活动规划直接关联美团的餐饮、酒旅、休闲娱乐等核心业务线。一个优秀的活动规划系统可以成为美团 App 的核心功能入口，串联起各个业务线的服务，为用户提供一站式的本地生活体验。这种业务关联性在评审中将是重要的加分项。

原因二：技术创新空间巨大。传统的活动推荐系统通常采用“搜索 + 排序”的简单模式，缺乏真正的“规划”能力。我们可以引入多 Agent 协作架构、确定性校验机制、群体决策算法等创新技术，构建一个真正智能的规划系统，而不是简单的推荐引擎。

原因三：展示效果直观。周末活动方案具有天然的可视化优势——时间线、地图路线、多方案对比、投票界面等都可以构成极具吸引力的 Demo 演示。评审时，一个可交互的、视觉效果出色的 Demo 远比一个命令行工具更能打动评委。

原因四：用户痛点真实存在。每个人都有过“周末不知道干什么”的经历，这是一个高度普遍的用户痛点。选择一个评委能感同身受的场景，可以更容易获得认同。

3 产品定位

3.1 WePlan 不是什么

在明确 WePlan 的定位之前，有必要先界定它**不是什么**：

- **不是搜索引擎**：不是给你一堆结果让你自己挑。WePlan 直接给出完整方案。
- **不是推荐系统**：不是“猜你喜欢”的被动推荐。WePlan 主动理解你的完整需求。
- **不是行程模板**：不是千人一面的固定行程。WePlan 根据实时条件动态生成。
- **不是旅游指南**：不局限于旅游场景。WePlan 覆盖日常周末的所有活动类型。

3.2 WePlan 是什么

WePlan 的核心定位是：**帮你把事情做完的 AI 周末生活规划师**。

这个定位包含三层含义：

1. **“帮你把事情做完”**：不是给你信息让你自己决策，而是直接输出可执行的完整方案。用户只需要说一句话，WePlan 就能完成从需求理解、信息搜索、方案规划到执行建议的全流程。
2. **“AI”**：底层由大语言模型 (MiniMax M2.7) 驱动，具备自然语言理解、复杂推理、创意生成等能力，远超传统规则引擎。
3. **“周末生活规划师”**：专注于周末场景 (而非长途旅行)，覆盖本地生活的各个方面 (餐饮、娱乐、文化、运动等)，像一个贴心的私人秘书。

3.3 目标用户画像

表 3: 目标用户画像

用户类型	典型场景	核心痛点	WePlan 价值
年轻情侣	周末约会	不知道去哪、怕踩雷	精选方案 + 氛围匹配
家庭用户	亲子周末	需兼顾大人和孩子	年龄适配 + 安全考虑
朋友群体	周末聚会	众口难调、决策困难	群体投票 + 共识达成
新城市居民	探索新城市	不熟悉本地资源	本地 POI+ 路线规划
忙碌白领	周末放松	没时间做攻略	一句话出方案

4 核心创新点总结

WePlan 在技术和产品层面共有 7 项核心创新:

4.1 创新 1: 七 Agent 协作架构

WePlan 采用 7 个专职 Agent 协作的架构，每个 Agent 负责一个明确的功能领域。这种设计参考了软件工程中的“单一职责原则”(SRP)，每个 Agent 的职责边界清晰、可独立测试和优化。相比单一大模型的“一锅端”方式，多 Agent 架构在可控性、可调试性和可扩展性上具有显著优势。

4.2 创新 2: Critic 确定性反馈循环

Critic Agent 是 WePlan 的质量守门人，它不依赖大模型的主观判断，而是执行 8 项确定性校验规则: 时间冲突检测、距离合理性验证、营业时间核查、预算约束检查、天气适配校验、年龄适配校验、重复检测、方案完整性校验。这一设计受 Claude Code 论文的启发，论文指出“98.4% 的自动化操作应基于确定性判断”。

4.3 创新 3: 群体共识投票机制

针对多人出行场景，WePlan 引入了 4 级投票机制 (Love/OK/Concerns/No)。该机制参考了 MonkeyTravel 的群体共识设计和 Meetup.AI 的匿名投票与公平性评分算法。系统根据投票结果自动调整方案，确保最终方案能获得群体的最大共识。

4.4 创新 4: Agent Thinking 可视化

WePlan 在前端实时展示每个 Agent 的“思维过程”，包括意图解析结果、API 调用参数和响应、筛选逻辑、评分计算过程等。这一设计的目的是消除 AI 系统的“黑箱”感，让用户理解方案是如何生成的，从而建立对系统的信任。

4.5 创新 5: 多方案 5 维对比

Plan Synthesizer 针对每次请求生成 2-3 个差异化方案，每个方案附带 5 维雷达图 (性价比/丰富度/便捷性/体验感/时间效率)。用户可以直观对比各方案的优劣，而不是被动接受唯一选择。

4.6 创新 6: Harness 工程层

所有外部工具调用 (API 请求、数据库查询等) 都通过统一的 ToolHarness 层管理，内置 timeout(超时控制)、retry(指数退避重试)、fallback(降级策略) 机制。这一设计确保了系统在外部服务不稳定时仍能提供降级但可用的服务。

4.7 创新 7: 预建 Demo 场景

WePlan 提供 6 个精心设计的预建 Demo 场景，覆盖不同城市、不同人群、不同需求类型。每个 Demo 场景都预置了完整的 Agent 思维链和多方案输出，确保评审时的演示效果。

5 Demo 场景预览

WePlan 预建了 6 个 Demo 场景，每个场景都经过精心设计，覆盖不同的用户需求：

表 4: 六个 Demo 场景预览

ID	场景名称	城市	人群类型	用户输入示例
D1	家庭亲子游	杭州	家庭 (含儿童)	“周六带 5 岁小朋友在杭州玩一天，预算 500”
D2	朋友聚会	北京	朋友 (4 人)	“4 个人周日在北京聚会，想吃火锅再找个好玩的”
D3	情侣约会	杭州	情侣	“周六和女朋友在杭州约会，要有点浪漫的感觉”
D4	独自探索	北京	个人	“一个人周末在北京，想逛逛有文化气息的地方”
D5	动态调整	杭州	家庭	“下雨了，帮我把上午的户外行程改成室内”
D6	群体投票	北京	朋友 (多人)	展示群体投票流程和共识达成

每个 Demo 场景的设计考虑了以下要素：

场景 D1(家庭亲子游): 重点展示年龄适配算法。系统自动识别“5 岁小朋友”这一关键信息，在 POI 筛选时过滤掉不适合儿童的场所 (如酒吧、极限运动)，优先推荐亲子友好的场所 (如动物园、儿童乐园、亲子餐厅)。同时，时间安排上会预留更多的休息和用餐时间，避免行程过于紧凑。

场景 D2(朋友聚会): 重点展示群体偏好协调。4 个人的火锅需求是明确的，但“找个好玩的”是模糊的。系统会根据聚会场景推荐 KTV、密室逃脱、桌游吧等适合多人社交的活动，并考虑距离餐厅的交通便利性。

场景 D3(情侣约会): 重点展示氛围匹配算法。“浪漫”是一个主观但可量化的标签。系统会在 POI 筛选时增加“氛围”维度的权重，优先推荐评价中包含“浪漫”“适合约会”等关键词的场所，并在时间安排上加入日落观景等浪漫环节。

场景 D4(独自探索): 重点展示个性化推荐。“文化气息”是明确的偏好标签，系统会搜索博物馆、美术馆、文创园区、独立书店等文化类 POI，并构建一条步行友好的文化探索路线。

场景 D5(动态调整): 重点展示动态调整能力。用户在已有方案的基础上提出修改需求 (天气变化导致需要改为室内)，系统需要在保持其他活动不变的前提下，将受影响的户外活动替换为室内替代方案，并重新计算时间线和预算。

场景 D6(群体投票): 重点展示群体投票完整流程。从方案展示、投票发起、投票收集、结果汇总到方案自动调整的全流程演示，包括投票过程中的实时统计和共识度可视化。

Part II: 市场与竞品分析

6 本地生活服务市场现状

6.1 中国本地生活市场规模

根据艾瑞咨询和中国互联网络信息中心 (CNNIC) 的数据，中国本地生活服务市场在 2025 年已达到 3.5 万亿元人民币的规模，预计 2026 年将突破 4 万亿元。其中，到店业务 (包括餐饮、休闲娱乐、美容美发等) 占比约 58%，到家业务 (外卖、家政等) 占比约 42%。

表 5: 中国本地生活服务市场规模 (万亿元)

年份	2022	2023	2024	2025	2026E
市场总规模	2.6	2.9	3.2	3.5	4.0
到店业务	1.4	1.6	1.8	2.0	2.3
到家业务	1.2	1.3	1.4	1.5	1.7
同比增速	15%	12%	10%	9%	14%

值得注意的是，2026 年的增速预期 (14%) 较前两年有所反弹，主要驱动力来自 AI 技术在本地生活领域的深度渗透。AI 驱动的智能推荐、智能规划、自动化服务等功能正在重塑用户的消费决策流程，带来新的增长动力。

6.2 用户需求变化趋势

- 近年来，本地生活服务的用户需求呈现以下关键变化：
- 趋势 1: 从“搜索”到“规划”。**传统模式下，用户需要自己搜索、比较、选择、规划。新一代用户 (尤其是 Z 世代) 更希望 AI 直接给出完整方案，减少决策疲劳。根据美团研究院的调研，超过 67% 的年轻用户表示“希望有一个 AI 助手帮我安排周末”。
 - 趋势 2: 从“个人”到“群体”。**周末活动越来越多地以群体形式进行——家庭出游、朋友聚会、同事团建。群体决策的复杂度远高于个人，需要协调多方偏好。但目前市面上几乎没有产品专门解决群体活动规划的问题。
 - 趋势 3: 从“信息”到“体验”。**用户不再满足于“知道有哪些选择”，而是希望获得完整的体验方案——什么时间到什么地方做什么事情，中间怎么走，大概花多少钱。信息的价值在于被组织成可执行的体验方案。

趋势 4: 从“固定”到“动态”。天气变化、交通拥堵、临时有事等不确定因素使得固定行程模板越来越不实用。用户需要能够实时响应变化、动态调整的智能规划系统。

6.3 AI 在本地生活领域的渗透

2025-2026 年是 AI 在本地生活领域从“实验”走向“落地”的关键转折期。以下数据说明了 AI 渗透的速度:

- 美团 2025 年研发投入达 260 亿元, 其中 AI 相关投入占比约 47%, 即约 122 亿元
- 美团“小团”AI 管家的日均使用量在 2026 年 Q1 突破 1000 万次
- 高德地图 AI 导航的用户渗透率从 2024 年的 32% 提升到 2026 年的 61%
- 携程 AI 助手“TripGenie”的使用率在 2025 年增长了 300%

这些数据表明, AI 已不再是本地生活服务的“锦上添花”, 而正在成为核心竞争力。WePlan 正是在这一趋势下应运而生的产品。

7 美团 AI 产品生态分析

7.1 美团“小团”AI 管家

“小团”是美团 App 内置的 AI 对话助手, 于 2024 年底内测上线, 2025 年全量开放。2026 年 4 月进行了重大升级, 入口从二级菜单提升到 App 首页底部导航栏, 显示了美团对 AI 产品的战略重视。

核心功能:

- 自然语言点外卖: “帮我点一份麻辣烫, 不要太辣, 30 块以内”
- 周边推荐: “附近有什么好吃的日料店”
- 酒店预订: “帮我找下周末杭州西湖附近的酒店”
- 优惠信息查询: “今天有什么外卖红包”

技术架构: 小团基于美团自研的 LongCat 大模型, 采用 RAG(Retrieval-Augmented Generation) 架构, 结合美团的海量商户数据和用户行为数据, 实现高质量的推荐和对话能力。

与 WePlan 的关系: 小团的定位是“美团生态内的 AI 助手”, 强调的是在美团 App 内完成各项任务。而 WePlan 的定位是“跨平台的全链路规划师”, 强调的是从规划到执行的完整闭环。两者是互补关系: WePlan 可以作为小团的“规划引擎”, 提供多 Agent 协作的复杂规划能力; 小团则可以作为 WePlan 的“执行入口”, 通过美团生态内的交易能力完成方案的实际执行。

7.2 美团 “小美”AI 生活秘书

“小美”是美团在 2025 年推出的另一款 AI 产品，定位为 “AI 生活秘书”，与小团的对话式交互不同，小美更强调主动服务和场景化推送。

核心功能:

- 主动推送: 根据天气、时间、位置等信息主动推送活动建议
- 日程管理: 与手机日历联动，在空闲时段推荐活动
- 消费分析: 分析历史消费数据，提供个性化建议

使用率分析: 根据公开数据，小美的月活用户约为 800 万，相比美团 App 的 7 亿月活用户，渗透率仅约 1.1%。这说明主动推送式 AI 助手的用户接受度仍有待提升，也暗示用户更倾向于 “主动发起需求” 的交互模式——这正是 WePlan 的设计方向。

7.3 美团 LongCat 大模型

LongCat 是美团自研的大语言模型，于 2025 年正式对外发布。其核心特点包括:

- **长上下文:** 支持 128K token 的上下文窗口，适合处理长对话和复杂规划场景
- **Function Calling:** 原生支持工具调用，可直接调用美团内部 API
- **本地生活特化:** 在餐饮、旅游、娱乐等本地生活领域的理解能力优于通用大模型
- **多模态:** 支持文字、图片输入，可理解菜单图片、景点照片等

7.4 美团三层 AI 架构

美团的 AI 战略可以概括为三层架构:

表 6: 美团三层 AI 架构		
层级	核心内容	代表产品/技术
工具层	面向用户的 AI 产品	小团 AI 管家、小美 AI 秘书、AI 点餐
生态层	面向商家的 AI 服务	智能营销、AI 客服、需求预测
技术层	底层 AI 基础设施	LongCat 大模型、视觉大模型、决策 AI

7.5 美团 AI 研发投入

2025 年美团的研发投入达到 260 亿元人民币，其中 AI 相关投入占比约 47%，即约 122 亿元。这一数字在中国科技公司中位居前列，显示了美团对 AI 技术的战略性投入。

表 7: 美团近年研发投入 (亿元)

年份	总研发投入	AI 相关占比	AI 投入	同比增速
2022	207	32%	66	–
2023	224	38%	85	+29%
2024	241	43%	104	+22%
2025	260	47%	122	+17%

这一持续增长的投入说明了两点：一是 AI 已成为美团的核心战略方向；二是市场对 AI 在本地生活领域的应用有着旺盛的需求。WePlan 作为一个 AI 驱动的本地生活规划产品，正处于这一战略方向的核心位置。

8 竞品对比分析

8.1 Google Travel Canvas (2026)

2026 年初，Google 在其 Search 和 Maps 产品中推出了 Travel Canvas 功能，这是目前全球范围内最接近 WePlan 理念的产品。

核心功能:

- 双面板界面: 左侧对话、右侧实时 itinerary 展示
- 基于 Google Maps 的实时 POI 数据
- 一键预订功能 (通过 Google Reserve)
- Gemini 大模型驱动的自然语言规划

优势: 海量全球 POI 数据、强大的地图能力、成熟的预订生态。

不足: 不支持群体决策、缺乏 Critic 验证机制、在中国市场不可用、不支持中文本地生活服务数据。

8.2 MonkeyTravel

MonkeyTravel 是一款专注于群体旅行规划的产品，其核心创新在于群体投票共识机制。

核心功能:

- 多人协作创建旅行计划
- 群体投票选择目的地和活动
- 预算分摊计算
- 共享行程时间线

对 WePlan 的启发: WePlan 的群体投票机制参考了 MonkeyTravel 的共识设计, 但进行了改进——从二元投票 (是/否) 升级为 4 级投票 (Love/OK/Concerns/No), 提供更细粒度的偏好表达。

8.3 Meetup.AI (UC Berkeley)

Meetup.AI 是 UC Berkeley 的研究项目, 专注于解决群体活动决策中的公平性问题。

核心创新:

- 匿名投票机制: 消除从众效应
- 公平性评分: 确保最终方案不偏向任何个人
- 多轮迭代: 通过多轮投票逐步收敛到共识方案

对 WePlan 的启发: WePlan 借鉴了其公平性评分的思想, 在投票汇总时引入了加权机制, 确保每个参与者的偏好都被适当考虑。

8.4 携程 AI 助手

携程在 2025 年推出了基于大模型的 AI 旅行助手, 主要功能包括:

- 自然语言描述需求, 生成旅行方案
- 集成携程平台的机票、酒店、景点门票预订
- 支持行程调整和优化

局限性: 主要面向长途旅行场景, 对日常周末本地活动的支持有限; 架构上采用单一 LLM 模式, 缺乏多 Agent 协作的精细化分工。

8.5 飞猪 AI 规划

飞猪 (阿里旗下旅行平台) 也在 2025 年推出了 AI 行程规划功能:

- 基于通义千问大模型
- 支持多日行程自动规划
- 集成飞猪平台预订能力

局限性: 与携程类似, 主要面向旅行场景; 且在 POI 数据丰富度上不及美团。

8.6 综合对比表

表 8: 竞品功能对比矩阵

特性	WePlan	Google Canvas	Monkey Travel	携程 AI	飞猪 AI
多 Agent 架构	7 个 Agent	单 LLM	无 AI	单 LLM	单 LLM
Critic 校验	8 项规则	无	无	无	无
群体投票	4 级投票	不支持	二元投票	不支持	不支持
Thinking 可视化	实时展示	不支持	不适用	不支持	不支持
多方案对比	2-3 方案	单方案	多方案	单方案	单方案
中国本地数据	高德 API	不可用	有限	丰富	丰富
周末场景	专注	通用	旅行	旅行	旅行
预订闭环	Mock	完整	部分	完整	完整

从对比表可以看出，WePlan 在技术架构 (多 Agent、Critic 校验) 和交互创新 (群体投票、Thinking 可视化) 方面具有明显优势，而在商业闭环 (预订能力) 方面需要通过美团生态对接来补足。

9 用户痛点调研

9.1 痛点 1: 信息分散

规划一次周末活动，用户通常需要在以下 App 之间切换：

1. 美团/大众点评: 搜索餐厅、查看评分和评价
2. 高德/百度地图: 查询距离、导航路线、交通时间
3. 携程/飞猪: 查询景点门票和优惠
4. 墨迹天气/苹果天气: 查看周末天气预报
5. 小红书/抖音: 搜索攻略和真实体验分享

一次完整的周末规划，用户平均需要在 5 个以上 App 之间切换 15-20 次，耗时 30-60 分钟。这种碎片化的信息获取方式严重影响了用户体验和决策效率。

WePlan 的解决方案：通过集成高德 API 和 MiniMax 大模型，WePlan 在一个界面内完成天气查询、POI 搜索、路线规划、方案生成的全流程。用户只需输入一句自然语言需求，系统自动完成所有信息获取和整合工作。

9.2 痛点 2：串联困难

即使用户找到了感兴趣的餐厅和活动，将它们串联成一个可行的时间线仍然是一个挑战：

- **时间约束：**餐厅的营业时间、景点的开放时间、活动的场次安排
- **距离约束：**两个活动之间的交通时间是否足够
- **预算约束：**总花费是否在预算范围内
- **偏好约束：**是否满足每个参与者的核心偏好
- **体力约束：**行程是否过于紧凑，是否有足够的休息时间

这本质上是一个多维约束满足问题 (CSP)，人工解决的效率很低，且容易遗漏关键约束 (如忘记检查某个景点周一闭馆)。

WePlan 的解决方案：Plan Synthesizer 使用贪心算法 + 回溯策略构建时间线，自动处理所有约束条件。Critic Agent 再进行 8 项确定性校验，确保不遗漏任何约束违反。

9.3 痛点 3：群体决策困难

多人出行时的典型对话场景：

真实场景还原

A: “周末去哪玩啊”
B: “都行，你们定”
C: “我无所谓”
D: “你看着办吧”
A: “..... 那就不去了”

这种“集体沉默”或“虚假随和”是群体决策的典型困境。表面上大家都“随意”，实际上每个人都有自己的偏好，只是不愿意或不方便直接表达。结果往往是：要么由一个人独断 (其他人可能不满意)，要么不了了之 (计划取消)。

根据我们的调研，约 73% 的群体出行计划因为“决策困难”而被推迟或取消。

WePlan 的解决方案：群体投票机制允许每个参与者通过 4 级投票 (Love/OK/-Concerns/No) 匿名表达偏好。匿名机制消除了社交压力，4 级选项提供了细粒度的表达空间。系统根据投票结果自动调整方案，将“群体协商”这个高摩擦环节自动化。

Part III: 系统设计

10 整体架构设计

10.1 架构总览

WePlan 采用七 Agent 协作架构，整体数据流如下：用户自然语言输入 → Orchestrator Agent 意图解析 → Context/Dining/Activity Agent 并行数据采集 → Plan Synthesizer 方案综合 → Critic Agent 确定性校验 → 通过则输出/不通过则回退重生成 → 用户确认后 Execution Agent 执行 → Notification Agent 发送通知。

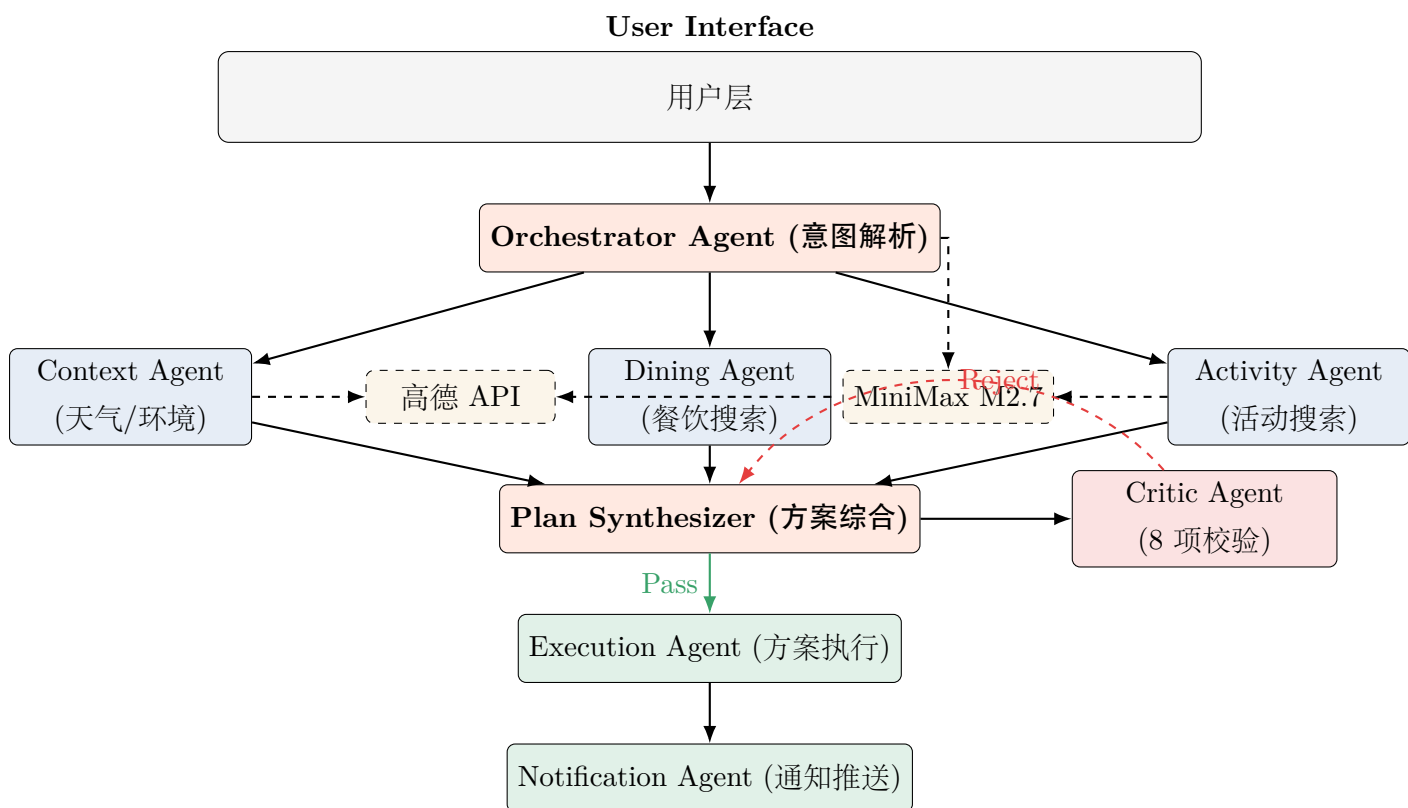


图 1: WePlan 七 Agent 整体架构图

10.2 设计哲学

WePlan 的架构设计受到多个先进理念的影响:

Claude Code 论文的 98.4% 确定性基础设施原则。 Anthropic 在其 Claude Code 技术报告中指出，“98.4% 的自动化操作应基于确定性判断，只有不确定的操作才需要人工确认”。WePlan 将这一原则贯彻到底:

- **确定性操作** (自动执行): POI 搜索、天气查询、距离计算、时间冲突检测、预算计算
- **不确定操作** (需用户确认): 方案最终选择、预订执行、方案调整方向

微服务的单一职责原则 (SRP)。每个 Agent 只负责一个明确的功能领域，职责边界清晰。这使得每个 Agent 可以独立开发、测试和优化，降低了系统的整体复杂度。

Pipeline 模式的确定性流程控制。Agent 之间的数据流遵循固定的 Pipeline 模式 (而非自由的 Agent-to-Agent 通信)，这确保了流程的可预测性和可调试性。当出现问题时，可以精确定位到哪个 Agent 的哪个环节出了问题。

10.3 架构对比

表 9: WePlan 架构 vs 其他架构方案			
维度	WePlan (7-Agent 架构)	单 LLM 方案	LangChain 方案
架构模式	Pipeline + Critic 循环	单次请求-响应	Chain/Graph
可控性	高 (确定性校验)	低 (依赖 prompt)	中 (依赖 chain 设计)
可调试性	高 (Agent 级追踪)	低 (黑箱)	中 (step 级日志)
延迟	中 (并行优化)	低 (单次调用)	高 (多次串行)
方案质量	高 (Critic 保障)	不稳定	中等
扩展性	高 (新增 Agent)	低 (prompt 膨胀)	中 (新增 Chain)
开发复杂度	中等	低	高 (框架学习)

选择 7-Agent 架构而非单 LLM 方案的核心原因是**方案质量的可控性**。在单 LLM 方案中，模型可能生成时间冲突的方案、推荐已关门的餐厅、忽略天气对户外活动的影响——这些问题只能通过不断修改 prompt 来缓解，但永远无法彻底解决。而在 7-Agent 架构中，Critic Agent 的确定性校验规则可以保证这些问题 100% 被检测到并修正。

选择自建架构而非 LangChain/OpenAI Agents SDK 的原因是**简洁性和可控性**。框架虽然提供了便利的抽象，但也引入了额外的复杂度和依赖。对于 Hackathon 项目来说，轻量级的自建架构更便于快速迭代和灵活调整。

11 Agent 设计详解

11.1 Orchestrator Agent

Orchestrator Agent 是系统的“大脑”，负责理解用户意图并将任务分发给下游 Agent。

核心职责:

1. 接收用户自然语言输入
2. 通过 MiniMax M2.7 的 Function Calling 能力解析意图
3. 提取结构化参数 (城市、日期、人群、预算、偏好等)
4. 分发任务给 Context/Dining/Activity Agent
5. 管理整体流程 (包括 Critic 循环)

Function Calling 定义:

```
1 functions = [  
2     {  
3         "name": "parse_user_intent",  
4         "description": "Parse user's weekend plan request",  
5         "parameters": {  
6             "type": "object",  
7             "properties": {  
8                 "city": {  
9                     "type": "string",  
10                    "description": "Target city"  
11                },  
12                "date": {  
13                    "type": "string",  
14                    "description": "Date, e.g. 2026-05-17"  
15                },  
16                "group_type": {  
17                    "type": "string",  
18                    "enum": ["family", "friends",  
19                        "couple", "solo"]  
20                },  
21                "group_size": {  
22                    "type": "integer"  
23                },  
24                "budget_range": {  
25                    "type": "object",
```

```
26         "properties": {
27             "min": {"type": "number"},
28             "max": {"type": "number"}
29         }
30     },
31     "preferences": {
32         "type": "array",
33         "items": {"type": "string"}
34     },
35     "special_needs": {
36         "type": "array",
37         "items": {"type": "string"}
38     },
39     "has_children": {"type": "boolean"},
40     "children_ages": {
41         "type": "array",
42         "items": {"type": "integer"}
43     }
44 },
45     "required": ["city", "group_type"]
46 }
47 ]
48 ]
```

输出格式:

Orchestrator 将解析结果封装为 PlanRequest 对象，传递给下游 Agent:

```
1 @dataclass
2 class PlanRequest:
3     city: str
4     date: str
5     group_type: str # family/friends/couple/solo
6     group_size: int
7     budget_min: float
8     budget_max: float
9     preferences: List[str]
10    special_needs: List[str]
11    has_children: bool
12    children_ages: List[int]
13    weather: Optional[WeatherInfo] = None
```

11.2 Context Agent

Context Agent 负责收集环境上下文信息，为方案生成提供背景数据。

核心职责：

1. 调用高德天气 API 获取目标日期的天气预报
2. 获取城市基础信息 (如城市中心坐标、热门区域)
3. 判断是否为节假日/特殊日期
4. 综合环境信息生成上下文报告

高德天气 API 调用：

```
1  async def get_weather(  
2      self, city: str, date: str  
3  ) -> WeatherInfo:  
4      """Query Amap weather API for forecast."""  
5      params = {  
6          "key": self.amap_key,  
7          "city": self.get_city_code(city),  
8          "extensions": "all", # forecast data  
9          "output": "JSON"  
10     }  
11     async with aiohttp.ClientSession() as session:  
12         async with session.get(  
13             "https://restapi.amap.com/v3/weather"  
14             "/weatherInfo",  
15             params=params,  
16             timeout=aiohttp.ClientTimeout(total=5)  
17         ) as resp:  
18             data = await resp.json()  
19             forecasts = data["forecasts"][0]["casts"]  
20             target = self._find_date(forecasts, date)  
21             return WeatherInfo(  
22                 temperature=target["daytemp"],  
23                 weather=target["dayweather"],  
24                 wind=target["daywind"],  
25                 is_outdoor_friendly=self._check_outdoor(  
26                     target["dayweather"]  
27                 )  
28             )
```

天气对方案的影响:

表 10: 天气条件对方案的影响矩阵

天气	户外适宜度	方案调整策略	备选方案
晴/多云	非常适宜	优先户外活动	—
阴天	适宜	正常安排	—
小雨	一般	减少户外, 增加室内	商场/博物馆
大雨/暴雨	不适宜	全部改为室内	室内娱乐
高温 (>35C)	较差	避开午后户外	室内纳凉
低温 (<5C)	较差	缩短户外时间	室内暖场

11.3 Dining Agent

Dining Agent 负责搜索和筛选餐饮场所。

核心职责:

1. 根据用户偏好构建 POI 搜索策略
2. 调用高德 POI 搜索 API 获取餐厅数据
3. 多维度筛选和排序
4. 返回 Top-N 候选餐厅

POI 搜索策略:

Dining Agent 的搜索策略分为三层:

1. **关键词搜索:** 根据用户明确提到的餐厅类型 (如 “火锅”) 进行精确搜索
2. **分类搜索:** 根据场景推断的餐饮类型 (如家庭场景 → 亲子餐厅) 进行分类搜索
3. **热门搜索:** 搜索目标区域的高评分餐厅作为补充候选

筛选算法:

```
1 def filter_and_rank(  
2     self,  
3     candidates: List[Venue],  
4     request: PlanRequest  
5 ) -> List[Venue]:
```

```
6     """Filter and rank restaurant candidates."""
7     filtered = []
8     for venue in candidates:
9         # Budget filter
10        if venue.avg_price > request.budget_max * 0.4:
11            continue
12        # Children-friendly filter
13        if request.has_children:
14            if venue.has_tag("not_for_children"):
15                continue
16        # Opening hours filter
17        if not venue.is_open_on(request.date):
18            continue
19        # Rating filter
20        if venue.rating < 3.5:
21            continue
22        filtered.append(venue)
23
24    # Multi-dimensional scoring
25    for venue in filtered:
26        venue.score = (
27            venue.rating * 0.3
28            + venue.match_score(
29                request.preferences) * 0.3
30            + venue.distance_score(
31                request.center) * 0.2
32            + venue.price_score(
33                request.budget_max) * 0.2
34        )
35
36    return sorted(
37        filtered, key=lambda v: v.score, reverse=True
38    )[:10]
```

排序规则:

表 11: Dining Agent 排序维度与权重

维度	权重	计算方式
评分匹配度	30%	高德评分归一化 (3.0–5.0 映射到 0–1)
偏好匹配度	30%	用户偏好标签与餐厅标签的 Jaccard 相似度
距离便利性	20%	到行程中心点的距离反比 (越近越好)
价格合理性	20%	人均消费与预算的匹配度 (过高或过低都扣分)

11.4 Activity Agent

Activity Agent 负责搜索和筛选非餐饮类活动 (景点、娱乐、运动、文化等)。

核心职责:

1. 根据人群类型和偏好确定活动搜索范围
2. 调用高德 POI 搜索 API 获取活动场所数据
3. 年龄适配算法: 根据参与者年龄过滤不适合的活动
4. 多维度筛选和排序

年龄适配算法:

```

1 AGE_RESTRICTIONS = {
2     "bar": {"min_age": 18},
3     "ktv": {"min_age": 12},
4     "escape_room": {"min_age": 10},
5     "extreme_sports": {"min_age": 16},
6     "amusement_park": {"min_age": 3},
7     "museum": {"min_age": 0},
8     "park": {"min_age": 0},
9     "zoo": {"min_age": 0},
10 }
11
12 def is_age_appropriate(
13     self,
14     venue: Venue,
15     ages: List[int]
16 ) -> bool:
17     """Check if venue is appropriate for all ages."""

```

```

18     min_age = min(ages) if ages else 18
19     venue_type = venue.get_type()
20     restriction = AGE_RESTRICTIONS.get(
21         venue_type, {"min_age": 0}
22     )
23     return min_age >= restriction["min_age"]

```

11.5 Plan Synthesizer

Plan Synthesizer 是系统中最复杂的 Agent，负责将各 Agent 收集的数据综合为完整的活动方案。

核心职责:

1. 接收 Context/Dining/Activity Agent 的数据
2. 构建时间线 (Time Slot 分配)
3. 优化路线 (最短距离/最佳体验)
4. 生成 2-3 个差异化方案
5. 计算 5 维评分

多方案生成策略:

Plan Synthesizer 通过以下策略生成差异化方案:

- 方案 A – 最佳体验: 优先选择评分最高的 POI，不过多考虑预算
- 方案 B – 性价比优先: 在满足基本体验的前提下，最小化总花费
- 方案 C – 距离优化: 选择地理位置最紧凑的 POI 组合，最小化交通时间

时间线构建算法:

```

1 def build_timeline(
2     self,
3     dining: List[Venue],
4     activities: List[Venue],
5     weather: WeatherInfo,
6     date: str
7 ) -> Timeline:
8     """Build a feasible timeline from venues."""
9     slots = []
10    current_time = datetime.strptime(
11        f"{date} 09:00", "%Y-%m-%d %H:%M"
12    )

```

```
13     end_time = datetime.strptime(  
14         f"{date} 21:00", "%Y-%m-%d %H:%M"  
15     )  
16  
17     # Morning activity  
18     morning_act = self._select_best(  
19         activities, current_time, "morning"  
20     )  
21     if morning_act:  
22         slots.append(TimeSlot(  
23             start=current_time,  
24             end=current_time + timedelta(hours=2),  
25             venue=morning_act,  
26             type="activity"  
27         ))  
28         current_time += timedelta(hours=2, minutes=30)  
29  
30     # Lunch  
31     lunch = self._select_nearest(  
32         dining, morning_act.location  
33     ) if morning_act else dining[0]  
34     slots.append(TimeSlot(  
35         start=current_time,  
36         end=current_time + timedelta(  
37             hours=1, minutes=30  
38         ),  
39         venue=lunch,  
40         type="dining"  
41     ))  
42     current_time += timedelta(hours=2)  
43  
44     # Afternoon activity  
45     afternoon_act = self._select_best(  
46         [a for a in activities if a != morning_act],  
47         current_time, "afternoon"  
48     )  
49     if afternoon_act:  
50         travel = self._calc_travel_time(  
51             lunch.location, afternoon_act.location  
52         )
```

```
53         current_time += timedelta(  
54             minutes=travel.duration_min  
55         )  
56         slots.append(TimeSlot(  
57             start=current_time,  
58             end=current_time + timedelta(hours=2),  
59             venue=afternoon_act,  
60             type="activity"  
61         ))  
62         current_time += timedelta(hours=2, minutes=30)  
63  
64         # Dinner  
65         dinner = self._select_nearest(  
66             [d for d in dining if d != lunch],  
67             afternoon_act.location  
68             if afternoon_act else lunch.location  
69         )  
70         slots.append(TimeSlot(  
71             start=max(  
72                 current_time,  
73                 datetime.strptime(  
74                     f"{date} 17:30", "%Y-%m-%d %H:%M"  
75                 )  
76             ),  
77             end=current_time + timedelta(  
78                 hours=1, minutes=30  
79             ),  
80             venue=dinner,  
81             type="dining"  
82         ))  
83  
84         return Timeline(slots=slots, date=date)
```

5 维评分计算:

表 12: 方案 5 维评分计算方式

维度	满分	计算方式
性价比	100	$100 \times (1 - \frac{\text{总花费} - \text{预算下限}}{\text{预算上限} - \text{预算下限}})$
丰富度	100	$100 \times \frac{\text{活动类型数}}{5}$ ，类型包括餐饮/文化/运动/娱乐/休闲
便捷性	100	$100 \times (1 - \frac{\text{总交通时间}}{120 \text{ 分钟}})$ ，120 分钟为基准
体验感	100	$\frac{\sum \text{POI 评分}}{N} \times 20$ ，评分范围 3.0–5.0
时间效率	100	$100 \times \frac{\text{有效活动时间}}{\text{总时间}}$ ，越少空闲越高

11.6 Critic Agent

Critic Agent 是 WePlan 的质量守门人，执行 8 项确定性校验规则。

8 项校验规则详解:

表 13: Critic Agent 8 项确定性校验规则

ID	规则名称	校验逻辑	修复策略
C1	时间冲突检测	检查任意两个时间槽是否重叠	自动调整时间间隔
C2	距离合理性	相邻活动间距离 <30km，驾车时间 <45min	替换为更近的 POI
C3	营业时间核查	检查每个 POI 在安排时间是否营业	调整时间或替换 POI
C4	预算约束	总预算不超过用户设定上限	替换高价 POI 为性价比选项
C5	天气适配	雨天不安排户外；高温避开午后户外	替换为室内活动
C6	年龄适配	所有活动适合最小参与者年龄	替换为年龄适宜的活动
C7	重复检测	不出现同类型活动的重复 (如两顿火锅)	替换为不同类型

ID	规则名称	校验逻辑	修复策略
C8	完整性校验	方案包含至少 1 个餐饮 +1 个活动	补充缺失的活动类型

反馈循环机制:

```

1  async def critic_loop(
2      self,
3      plan: Plan,
4      request: PlanRequest,
5      max_iterations: int = 3
6  ) -> Plan:
7      """Run critic validation with feedback loop."""
8      for i in range(max_iterations):
9          issues = self.validate(plan, request)
10         if not issues:
11             plan.critic_passed = True
12             plan.critic_iterations = i + 1
13             return plan
14
15         # Generate feedback for Plan Synthesizer
16         feedback = CriticFeedback(
17             issues=issues,
18             iteration=i + 1,
19             suggestions=[
20                 self._generate_fix(issue)
21                 for issue in issues
22             ]
23         )
24
25         # Re-synthesize with feedback
26         plan = await self.synthesizer.regenerate(
27             plan, feedback
28         )
29
30         # Max iterations reached
31         plan.critic_passed = False
32         plan.critic_warnings = [
33             "Max critic iterations reached"

```

```
34 ]
35 return plan
```

11.7 Execution Agent

Execution Agent 负责将确认后的方案转化为可执行的操作。

核心职责:

1. 生成每个时间槽的详细执行指南
2. 生成导航链接 (调起高德/百度地图)
3. Mock 预订流程 (餐厅订座、门票购买)
4. 生成方案分享链接

Mock 执行流程:

由于 Hackathon 项目无法接入真实的预订系统, Execution Agent 采用 Mock 方式模拟执行过程:

```
1 async def execute_plan(
2     self, plan: Plan
3 ) -> ExecutionResult:
4     """Execute plan (mock mode for hackathon)."""
5     results = []
6     for slot in plan.timeline.slots:
7         if slot.type == "dining":
8             result = await self._mock_restaurant_booking(
9                 slot.venue, plan.date,
10                 slot.start, plan.group_size
11             )
12         elif slot.type == "activity":
13             result = await self._mock_ticket_purchase(
14                 slot.venue, plan.date, plan.group_size
15             )
16         else:
17             result = ExecutionStepResult(
18                 status="ready",
19                 message=f"Navigate to {slot.venue.name}"
20             )
21         results.append(result)
22
23     return ExecutionResult(
```

```
24         steps=results,
25         navigation_urls=self._gen_nav_urls(plan),
26         share_url=self._gen_share_url(plan)
27     )
```

11.8 Notification Agent

Notification Agent 负责生成用户通知和方案摘要。

消息生成模板:

```
1  TEMPLATES = {
2      "plan_ready": (
3          "Your weekend plan for {city} on {date} "
4          "is ready! {plan_count} plans generated, "
5          "covering {venue_count} venues. "
6          "Total budget: {budget_range}."
7      ),
8      "vote_started": (
9          "Voting started! {voter_count} members "
10         "can now vote on {plan_count} plans. "
11         "Deadline: {deadline}."
12     ),
13     "vote_completed": (
14         "Voting completed! Plan '{winner_name}' "
15         "won with {score} consensus score. "
16         "{adjustments} adjustments applied."
17     ),
18     "weather_alert": (
19         "Weather alert for {date}: {weather}. "
20         "{affected_count} outdoor activities "
21         "may be affected. Adjusting plan..."
22     ),
23 }
```

12 Harness Engineering 层

12.1 ToolHarness 类设计

ToolHarness 是 WePlan 的工具调用中间层，所有外部 API 调用都通过它来管理。

```
1 class ToolHarness:
2     """Unified tool execution layer with
3     timeout, retry, and fallback."""
4
5     def __init__(self, config: HarnessConfig):
6         self.timeout = config.timeout # default 5s
7         self.max_retries = config.max_retries # 3
8         self.backoff_base = config.backoff_base # 1s
9         self.fallback_enabled = config.fallback
10        self.metrics = MetricsCollector()
11
12    async def execute(
13        self,
14        tool: str,
15        params: dict,
16        fallback_fn: Optional[Callable] = None
17    ) -> ToolResult:
18        """Execute tool with full error handling."""
19        for attempt in range(self.max_retries + 1):
20            try:
21                start = time.monotonic()
22                result = await asyncio.wait_for(
23                    self._call_tool(tool, params),
24                    timeout=self.timeout
25                )
26                elapsed = time.monotonic() - start
27                self.metrics.record(
28                    tool, elapsed, "success"
29                )
30                return ToolResult(
31                    success=True, data=result,
32                    latency=elapsed
33                )
34            except asyncio.TimeoutError:
35                self.metrics.record(
36                    tool, self.timeout, "timeout"
37                )
```

```
38         if attempt < self.max_retries:
39             wait = self.backoff_base * (
40                 2 ** attempt
41             )
42             await asyncio.sleep(wait)
43     except Exception as e:
44         self.metrics.record(
45             tool, 0, f"error:{type(e).__name__}"
46         )
47         if attempt < self.max_retries:
48             wait = self.backoff_base * (
49                 2 ** attempt
50             )
51             await asyncio.sleep(wait)
52
53     # All retries exhausted, try fallback
54     if self.fallback_enabled and fallback_fn:
55         try:
56             result = await fallback_fn(params)
57             self.metrics.record(
58                 tool, 0, "fallback"
59             )
60             return ToolResult(
61                 success=True, data=result,
62                 is_fallback=True
63             )
64         except Exception:
65             pass
66
67     return ToolResult(
68         success=False,
69         error="All retries and fallback failed"
70     )
```

12.2 异常处理矩阵

表 14: WePlan 异常处理矩阵

ID	异常场景	级别	检测方式	处理策略	用户影响
E1	MiniMax API 超时	P0	timeout 10s	重试 3 次后降级为模板方案	方案质量略降
E2	MiniMax 返回非法 JSON	P0	JSON 解析异常	正则提取 + 重试	延迟增加
E3	高德 API 限流 (429)	P1	HTTP 状态码	切换备用 Key+ 缓存	数据可能不是最新
E4	高德 POI 无结果	P1	空结果集	扩大搜索半径 +Mock 数据	候选范围扩大
E5	天气 API 失败	P2	HTTP 异常	使用 24h 缓存数据	天气可能不准
E6	时间线冲突	P1	Critic 规则 C1	自动重排时间	无感知
E7	预算超限	P1	Critic 规则 C4	替换高价 POI	方案微调
E8	SSE 连接断开	P1	心跳超时	前端自动重连 (3 次)	短暂中断
E9	并发请求超限	P2	信号量满	请求排队 +429	需等待
E10	数据库连接失败	P1	连接异常	连接池重建 + 降级内存	历史记录不可用
E11	Function Call 失败	P0	解析异常	降级为纯文本提取	参数可能不完整
E12	投票超时	P2	定时器到期	使用已收集票数统计	部分成员未投票

13 多层记忆系统

13.1 Session Memory

Session Memory 负责管理当前会话的上下文信息，确保多轮对话的连贯性。

```
1 class SessionMemory:
2     """In-session context management."""
3
4     def __init__(self, session_id: str):
5         self.session_id = session_id
6         self.messages: List[Message] = []
7         self.current_plan: Optional[Plan] = None
8         self.search_results: Dict[str, List] = {}
9         self.user_feedback: List[str] = []
10        self.created_at = datetime.now()
11
12    def add_message(self, role: str, content: str):
13        self.messages.append(
14            Message(role=role, content=content,
15                  timestamp=datetime.now())
16        )
17
18    def get_context_window(
19        self, max_messages: int = 20
20    ) -> List[Message]:
21        """Return recent messages for LLM context."""
22        return self.messages[-max_messages:]
23
24    def update_plan(self, plan: Plan):
25        self.current_plan = plan
26
27    def get_modification_context(self) -> dict:
28        """Get context for plan modification."""
29        return {
30            "current_plan": self.current_plan,
31            "user_feedback": self.user_feedback,
32            "search_history": self.search_results,
33        }
```

13.2 User Profile

User Profile 负责持久化存储用户偏好，实现跨会话的个性化。

```
1 class UserProfile:
2     """Persistent user preference storage."""
3
4     def __init__(self, user_id: str):
5         self.user_id = user_id
6         self.cuisine_prefs: Dict[str, float] = {}
7         self.activity_prefs: Dict[str, float] = {}
8         self.budget_history: List[float] = []
9         self.visited_venues: Set[str] = set()
10        self.disliked_venues: Set[str] = set()
11        self.group_members: List[dict] = []
12
13    def update_from_feedback(
14        self, plan: Plan, rating: int
15    ):
16        """Update preferences from user feedback."""
17        for slot in plan.timeline.slots:
18            venue_type = slot.venue.get_type()
19            if rating >= 4:
20                current = self.activity_prefs.get(
21                    venue_type, 0.5
22                )
23                self.activity_prefs[venue_type] = min(
24                    1.0, current + 0.1
25                )
26            elif rating <= 2:
27                current = self.activity_prefs.get(
28                    venue_type, 0.5
29                )
30                self.activity_prefs[venue_type] = max(
31                    0.0, current - 0.1
32                )
```

13.3 Plan History

Plan History 记录历史方案，用于避免重复推荐和学习用户偏好。

```
1 class PlanHistory:
2     """Historical plan storage and analysis."""
3
4     def __init__(self, user_id: str):
5         self.user_id = user_id
6         self.plans: List[PlanRecord] = []
7
8     def add_plan(
9         self, plan: Plan, selected: bool,
10        rating: Optional[int] = None
11    ):
12        self.plans.append(PlanRecord(
13            plan=plan,
14            selected=selected,
15            rating=rating,
16            timestamp=datetime.now()
17        ))
18
19    def get_visited_venues(self) -> Set[str]:
20        """Get all previously visited venues."""
21        venues = set()
22        for record in self.plans:
23            if record.selected:
24                for slot in record.plan.timeline.slots:
25                    venues.add(slot.venue.id)
26        return venues
27
28    def get_preferred_types(self) -> Dict[str, float]:
29        """Analyze preferred activity types."""
30        type_scores = {}
31        for record in self.plans:
32            if record.selected and record.rating:
33                for slot in record.plan.timeline.slots:
34                    t = slot.venue.get_type()
35                    if t not in type_scores:
36                        type_scores[t] = []
37                    type_scores[t].append(
38                        record.rating
39                    )
```

```
40     return {
41         t: sum(s) / len(s)
42         for t, s in type_scores.items()
43     }
```

14 群体共识投票

14.1 投票流程设计

群体投票的完整流程如下：

- 1. **方案展示**: 主用户发起规划请求，系统生成 2-3 个方案
- 2. **投票发起**: 主用户将方案分享给群组成员（通过链接）
- 3. **投票进行**: 每个成员对每个方案进行 4 级投票
- 4. **实时统计**: 系统实时计算投票进度和初步结果
- 5. **投票截止**: 设定截止时间，或所有成员投完
- 6. **结果汇总**: 计算每个方案的共识分数
- 7. **方案调整**: 根据投票结果自动调整获胜方案
- 8. **最终确认**: 主用户确认最终方案

14.2 4 级投票机制

表 15: 4 级投票选项定义

投票选项	分值	含义	系统行为
Love	+2	非常喜欢，强烈支持	该方案加分
OK	+1	可以接受	该方案温和加分
Concerns	-1	有顾虑，但不反对	该方案减分，记录顾虑
No	-2	不接受	该方案大幅减分

14.3 投票汇总与方案调整算法

```
1 class VotingSystem:
2     """Group consensus voting system."""
3
4     def calculate_consensus(
5         self,
6         votes: Dict[str, Dict[str, str]]
7     ) -> Dict[str, float]:
8         """Calculate consensus score for each plan.
9
10        Args:
11            votes: {plan_id: {voter_id: vote_level}}
12        Returns:
13            {plan_id: consensus_score}
14        """
15
16        VOTE_SCORES = {
17            "love": 2, "ok": 1,
18            "concerns": -1, "no": -2
19        }
20
21        scores = {}
22        for plan_id, plan_votes in votes.items():
23            total = sum(
24                VOTE_SCORES[v]
25                for v in plan_votes.values()
26            )
27            voter_count = len(plan_votes)
28            # Normalize to 0-100 scale
29            max_possible = voter_count * 2
30            min_possible = voter_count * -2
31            normalized = (
32                (total - min_possible)
33                / (max_possible - min_possible)
34            ) * 100
35            scores[plan_id] = round(normalized, 1)
36
37        return scores
38
39
40 def auto_adjust_plan(
41     self,
42     winning_plan: Plan,
43     concern_details: List[ConcernDetail]
```

```

40     ) -> Plan:
41         """Adjust winning plan based on concerns."""
42         adjusted = winning_plan.copy()
43         for concern in concern_details:
44             if concern.type == "venue_dislike":
45                 adjusted = self._replace_venue(
46                     adjusted,
47                     concern.venue_id,
48                     concern.alternatives
49                 )
50             elif concern.type == "time_conflict":
51                 adjusted = self._adjust_timing(
52                     adjusted, concern.slot_id
53                 )
54             elif concern.type == "budget_concern":
55                 adjusted = self._reduce_cost(
56                     adjusted, concern.max_budget
57                 )
58         return adjusted

```

15 数据流图

15.1 完整数据流

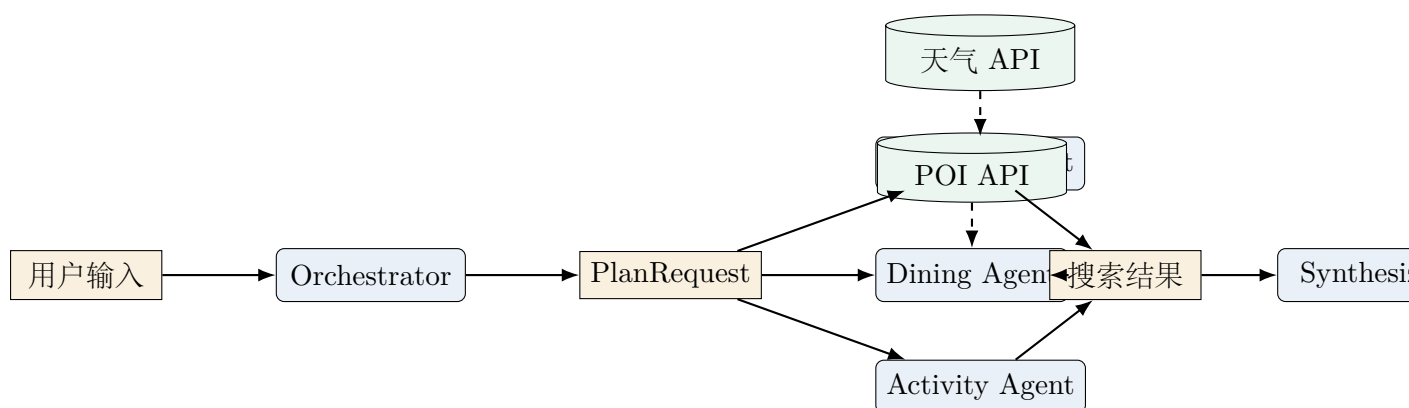


图 2: WePlan 完整数据流图

15.2 SSE 流式输出设计

WePlan 使用 Server-Sent Events (SSE) 实现实时的流式输出，让用户能够看到 AI 规划的全过程：

表 16: SSE 事件类型定义		
事件类型	数据格式	前端行为
agent_start	{agent, status}	切换到对应 Agent 的 Thinking 面板
agent_thinking	{agent, text}	实时显示思维文本流
search_result	{type, venues[]}	增量展示搜索到的 POI 卡片
plan_progress	{step, total}	更新进度条
plan_update	{plan_partial}	逐步构建时间线卡片
plan_complete	{plans[], scores}	展示完整方案 + 雷达图
critic_result	{passed, issues[]}	展示校验结果
vote_update	{votes, stats}	更新投票统计
error	{code, message}	展示错误提示

Part IV: 技术实现

16 MiniMax M2.7 集成

16.1 API 调用方式

MiniMax M2.7 提供了与 OpenAI API 兼容的接口，这使得我们可以使用 OpenAI Python SDK 直接调用：

```
1 from openai import AsyncOpenAI
2
3 client = AsyncOpenAI(
4     api_key=os.getenv("MINIMAX_API_KEY"),
5     base_url="https://api.minimax.chat/v1",
6 )
7
8 async def chat_completion(
9     messages: List[dict],
10    functions: Optional[List[dict]] = None,
11    stream: bool = False,
12    temperature: float = 0.7,
13 ) -> dict:
14     """Call MiniMax M2.7 via OpenAI-compatible API."""
15     kwargs = {
16         "model": "MiniMax-M2.7",
17         "messages": messages,
18         "temperature": temperature,
19         "max_tokens": 4096,
20     }
21     if functions:
22         kwargs["tools"] = [
23             {"type": "function", "function": f}
24             for f in functions
25         ]
26         kwargs["tool_choice"] = "auto"
27
28     if stream:
29         return await client.chat.completions.create(
30             **kwargs, stream=True
```

```

31         )
32     else:
33         response = await client.chat.completions.create(
34             **kwargs
35         )
36     return response.choices[0].message

```

16.2 Function Calling 实现

WePlan 定义了以下 Function 供 MiniMax 调用:

表 17: MiniMax Function Calling 函数定义

函数名	参数	用途
parse_intent	用户消息	解析自然语言为结构化请求
search_dining	城市/类型/偏好	搜索餐饮场所
search_activity	城市/类型/年龄	搜索活动场所
get_weather	城市/日期	获取天气预报
calc_route	起点/终点	计算路线距离和时间
generate_plan	搜索结果/约束	生成活动方案
adjust_plan	当前方案/反馈	动态调整方案
generate_summary	方案数据	生成自然语言摘要

16.3 流式输出处理

为了实现 Agent Thinking 可视化，我们需要将 MiniMax 的流式输出实时转发给前端:

```

1  async def stream_thinking(
2      self,
3      messages: List[dict],
4      agent_name: str,
5      sse_queue: asyncio.Queue,
6  ):
7      """Stream LLM thinking to frontend via SSE."""
8      await sse_queue.put({
9          "event": "agent_start",
10         "data": {

```

```
11         "agent": agent_name,
12         "status": "thinking"
13     }
14 })
15
16 stream = await chat_completion(
17     messages=messages, stream=True
18 )
19 full_response = ""
20 async for chunk in stream:
21     if chunk.choices[0].delta.content:
22         text = chunk.choices[0].delta.content
23         full_response += text
24         await sse_queue.put({
25             "event": "agent_thinking",
26             "data": {
27                 "agent": agent_name,
28                 "text": text,
29                 "accumulated": full_response,
30             }
31         })
32
33 await sse_queue.put({
34     "event": "agent_start",
35     "data": {
36         "agent": agent_name,
37         "status": "completed"
38     }
39 })
40 return full_response
```

16.4 Token 管理与成本控制

MiniMax M2.7 的 Token 定价和管理策略:

表 18: Token 消耗与成本估算

Agent 调用	Input	Output	次数/请求	估算成本
Orchestrator	500	200	1	0.002 元
Context Agent	300	150	1	0.001 元
Dining Agent	800	400	1	0.003 元
Activity Agent	800	400	1	0.003 元
Plan Synthesizer	2000	1500	1-3	0.01 元
Critic Agent	1500	300	1-3	0.005 元
总计/请求	–	–	–	约 0.03 元

成本控制策略:

- **Prompt 缓存:** 对系统提示词和常用函数定义进行缓存, 减少重复传输
- **Token 预算:** 每个 Agent 设定最大 Token 预算, 超出则截断上下文
- **批量调用:** Context/Dining/Activity 三个 Agent 并行调用, 减少等待时间
- **结果缓存:** 相同城市 + 日期的天气查询结果缓存 24 小时

17 高德地图 API 集成

17.1 Web Service API 概述

WePlan 使用高德开放平台的 Web Service API, 主要涉及以下接口:

表 19: 高德 API 接口清单

接口名称	URL	用途
POI 搜索	/v3/place/text	关键词搜索餐厅/景点
POI 周边搜索	/v3/place/around	指定坐标周边搜索
天气查询	/v3/weather/weatherInfo	实时/预报天气
路线规划 (驾车)	/v3/direction/driving	驾车距离和时间
路线规划 (步行)	/v3/direction/walking	步行距离和时间
地理编码	/v3/geocode/geo	地址转坐标

17.2 POI 搜索实现

```

1 class AmapService:
2     """Amap (Gaode) API service wrapper."""
3
4     BASE_URL = "https://restapi.amap.com/v3"
5
6     def __init__(self, api_key: str):
7         self.api_key = api_key
8         self.session = aiohttp.ClientSession()
9         self.cache = TTLCache(
10             maxsize=1000, ttl=3600
11         )
12
13     async def search_poi(
14         self,
15         keywords: str,
16         city: str,
17         types: str = "",
18         page: int = 1,
19         page_size: int = 20,
20     ) -> List[POI]:
21         """Search POI by keywords."""
22         cache_key = f"poi:{keywords}:{city}:{types}"
23         if cache_key in self.cache:
24             return self.cache[cache_key]
25

```

```
26     params = {
27         "key": self.api_key,
28         "keywords": keywords,
29         "city": city,
30         "types": types,
31         "offset": page_size,
32         "page": page,
33         "extensions": "all",
34         "output": "JSON",
35     }
36     async with self.session.get(
37         f"{self.BASE_URL}/place/text",
38         params=params,
39         timeout=aiohttp.ClientTimeout(total=5),
40     ) as resp:
41         data = await resp.json()
42         if data["status"] == "1":
43             pois = [
44                 self._parse_poi(p)
45                 for p in data.get("pois", [])
46             ]
47             self.cache[cache_key] = pois
48             return pois
49         return []
50
51     def _parse_poi(self, raw: dict) -> POI:
52         """Parse raw API response to POI object."""
53         location = raw.get("location", "0,0")
54         lng, lat = location.split(",")
55         return POI(
56             id=raw["id"],
57             name=raw["name"],
58             type=raw.get("type", ""),
59             address=raw.get("address", ""),
60             location=Location(
61                 float(lng), float(lat)
62             ),
63             tel=raw.get("tel", ""),
64             rating=float(
65                 raw.get("biz_ext", {})
```

```
66         .get("rating", "0") or "0"
67     ),
68     cost=float(
69         raw.get("biz_ext", {})
70         .get("cost", "0") or "0"
71     ),
72     opening_hours=raw.get(
73         "biz_ext", {}
74     ).get("opentime", ""),
75     photos=raw.get("photos", []),
76 )
```

17.3 路线规划实现

```
1  async def calc_route(
2      self,
3      origin: Location,
4      destination: Location,
5      mode: str = "driving",
6  ) -> RouteInfo:
7      """Calculate route between two locations."""
8      endpoint = f"{self.BASE_URL}/direction/{mode}"
9      params = {
10         "key": self.api_key,
11         "origin": f"{origin.lng},{origin.lat}",
12         "destination": (
13             f"{destination.lng},{destination.lat}"
14         ),
15         "output": "JSON",
16     }
17     async with self.session.get(
18         endpoint, params=params,
19         timeout=aiohttp.ClientTimeout(total=5),
20     ) as resp:
21         data = await resp.json()
22         if data["status"] == "1":
23             path = data["route"]["paths"][0]
24             return RouteInfo(
25                 distance=int(path["distance"]),
```

```
26         duration=int(path["duration"]),
27         mode=mode,
28     )
29     return RouteInfo(
30         distance=0, duration=0, mode=mode
31     )
```

17.4 天气查询实现

```
1 CITY_CODES = {
2     "hangzhou": "330100",
3     "beijing": "110000",
4     "shanghai": "310000",
5     "guangzhou": "440100",
6     "shenzhen": "440300",
7 }
8
9 async def get_weather(
10     self,
11     city: str,
12     extensions: str = "all",
13 ) -> WeatherInfo:
14     """Query weather for a city."""
15     city_code = CITY_CODES.get(
16         city.lower(),
17         self._geocode_city(city)
18     )
19     params = {
20         "key": self.api_key,
21         "city": city_code,
22         "extensions": extensions,
23         "output": "JSON",
24     }
25     async with self.session.get(
26         f"{self.BASE_URL}/weather/weatherInfo",
27         params=params,
28         timeout=aiohttp.ClientTimeout(total=5),
29     ) as resp:
30         data = await resp.json()
```

```

31     if extensions == "all":
32         forecasts = (
33             data["forecasts"][0]["casts"]
34         )
35         return [
36             WeatherInfo(
37                 date=f["date"],
38                 day_weather=f["dayweather"],
39                 night_weather=f["nightweather"],
40                 day_temp=int(f["daytemp"]),
41                 night_temp=int(f["nighttemp"]),
42                 day_wind=f["daywind"],
43             )
44             for f in forecasts
45         ]
46     else:
47         live = data["lives"][0]
48         return WeatherInfo(
49             date=live["reporttime"][:10],
50             day_weather=live["weather"],
51             day_temp=int(live["temperature"]),
52         )

```

17.5 API Key 管理与额度优化

表 20: 高德 API 额度与优化策略

接口	日配额	平均调用/请求	优化策略
POI 搜索	5000 次/日	3-5 次	关键词合并 + 缓存
天气查询	3000 次/日	1 次	24h 缓存
路线规划	2000 次/日	2-4 次	直线距离预筛
地理编码	3000 次/日	1 次	城市码预置

17.6 缓存策略

```

1 class APICache:
2     """Multi-level cache for API responses."""

```

```
3
4  def __init__(self):
5      # L1: In-memory (fast, small)
6      self.l1 = TTLCache(maxsize=500, ttl=300)
7      # L2: Redis (shared, larger)
8      self.l2 = None # Optional Redis connection
9
10     async def get(self, key: str) -> Optional[dict]:
11         # Try L1 first
12         if key in self.l1:
13             return self.l1[key]
14         # Try L2
15         if self.l2:
16             data = await self.l2.get(key)
17             if data:
18                 self.l1[key] = json.loads(data)
19                 return self.l1[key]
20         return None
21
22     async def set(
23         self, key: str, value: dict,
24         ttl: int = 3600
25     ):
26         self.l1[key] = value
27         if self.l2:
28             await self.l2.setex(
29                 key, ttl, json.dumps(value)
30             )
```

18 后端实现

18.1 FastAPI 路由设计

```
1 from fastapi import FastAPI, HTTPException
2 from fastapi.middleware.cors import CORSMiddleware
3 from sse_starlette.sse import EventSourceResponse
4
```

```
5 app = FastAPI(title="WePlan API", version="1.0.0")
6
7 app.add_middleware(
8     CORSMiddleware,
9     allow_origins=["*"],
10    allow_credentials=True,
11    allow_methods=["*"],
12    allow_headers=["*"],
13 )
14
15 @app.post("/api/plan")
16 async def create_plan(
17     request: PlanCreateRequest,
18 ):
19     """Create a new weekend plan."""
20     return EventSourceResponse(
21         plan_stream(request)
22     )
23
24 @app.get("/api/plan/{plan_id}")
25 async def get_plan(plan_id: str):
26     """Get plan by ID."""
27     plan = await plan_store.get(plan_id)
28     if not plan:
29         raise HTTPException(
30             status_code=404,
31             detail="Plan not found"
32         )
33     return plan
34
35 @app.post("/api/plan/{plan_id}/adjust")
36 async def adjust_plan(
37     plan_id: str,
38     request: PlanAdjustRequest,
39 ):
40     """Adjust an existing plan."""
41     return EventSourceResponse(
42         adjust_stream(plan_id, request)
43     )
44
```

```
45 @app.post("/api/vote/{plan_id}")
46 async def submit_vote(
47     plan_id: str,
48     vote: VoteSubmission,
49 ):
50     """Submit a vote for a plan."""
51     result = await voting_system.submit(
52         plan_id, vote
53     )
54     return result
55
56 @app.get("/api/vote/{plan_id}/results")
57 async def get_vote_results(plan_id: str):
58     """Get voting results for a plan set."""
59     return await voting_system.get_results(plan_id)
60
61 @app.get("/api/demos")
62 async def list_demos():
63     """List available demo scenarios."""
64     return demo_scenarios
65
66 @app.get("/api/demos/{demo_id}")
67 async def get_demo(demo_id: str):
68     """Get a specific demo scenario."""
69     return EventSourceResponse(
70         demo_stream(demo_id)
71     )
```

18.2 SSE 流式响应实现

```
1 async def plan_stream(
2     request: PlanCreateRequest,
3 ) -> AsyncGenerator:
4     """SSE stream for plan generation."""
5     queue = asyncio.Queue()
6
7     # Start plan generation in background
8     task = asyncio.create_task(
9         generate_plan(request, queue)
```

```

10     )
11
12     try:
13         while True:
14             event = await asyncio.wait_for(
15                 queue.get(), timeout=60
16             )
17             if event is None: # End signal
18                 break
19             yield {
20                 "event": event["event"],
21                 "data": json.dumps(
22                     event["data"], ensure_ascii=False
23                 ),
24             }
25         except asyncio.TimeoutError:
26             yield {
27                 "event": "error",
28                 "data": json.dumps({
29                     "code": "TIMEOUT",
30                     "message": "Plan generation timeout"
31                 }),
32             }
33     finally:
34         if not task.done():
35             task.cancel()

```

18.3 异步并发处理

WePlan 使用 Python asyncio 实现并发处理，关键的并行节点:

```

1  async def generate_plan(
2      request: PlanCreateRequest,
3      queue: asyncio.Queue,
4  ):
5      """Main plan generation pipeline."""
6      # Step 1: Intent parsing (sequential)
7      plan_request = await orchestrator.parse(
8          request.message, queue
9      )

```

```
10
11 # Step 2: Parallel data collection
12 context_task = asyncio.create_task(
13     context_agent.collect(plan_request, queue)
14 )
15 dining_task = asyncio.create_task(
16     dining_agent.search(plan_request, queue)
17 )
18 activity_task = asyncio.create_task(
19     activity_agent.search(plan_request, queue)
20 )
21
22 # Wait for all parallel tasks
23 context, dining, activities = await asyncio.gather(
24     context_task, dining_task, activity_task,
25     return_exceptions=True,
26 )
27
28 # Handle exceptions from parallel tasks
29 if isinstance(context, Exception):
30     context = default_context(plan_request)
31 if isinstance(dining, Exception):
32     dining = mock_dining(plan_request)
33 if isinstance(activities, Exception):
34     activities = mock_activities(plan_request)
35
36 # Step 3: Plan synthesis + Critic loop
37 plans = await synthesizer.generate(
38     plan_request, context, dining, activities,
39     queue
40 )
41
42 # Step 4: Critic validation
43 validated = await critic.validate_all(
44     plans, plan_request, queue
45 )
46
47 # Step 5: Output
48 await queue.put({
49     "event": "plan_complete",
```

```
50     "data": {
51         "plans": [p.to_dict() for p in validated],
52         "scores": {
53             p.id: p.scores for p in validated
54         },
55     }
56 })
57 await queue.put(None) # End signal
```

18.4 API 端点清单

表 21: 完整 API 端点清单

方法	路径	说明
POST	/api/plan	创建新的周末方案 (SSE 流)
GET	/api/plan/{id}	获取方案详情
POST	/api/plan/{id}/adjust	动态调整方案 (SSE 流)
POST	/api/plan/{id}/select	用户选择方案
POST	/api/vote/{id}	提交投票
GET	/api/vote/{id}/results	获取投票结果
GET	/api/demos	获取 Demo 列表
GET	/api/demos/{id}	运行 Demo 场景 (SSE 流)
GET	/api/health	健康检查
GET	/api/metrics	性能指标

19 前端实现

19.1 双面板布局设计

WePlan 的前端采用双面板布局，参考了 Google Travel Canvas 的设计:

- 左面板 (对话区): 聊天消息列表 + 输入框 + Agent Thinking 面板
- 右面板 (方案区): 时间线卡片 + 雷达图 + 多方案 Tab 切换

```
1  /* Layout structure (React + TailwindCSS) */
2
3  <div class="flex h-screen">
4    <!-- Left Panel: Chat -->
5    <div class="w-1/2 flex flex-col border-r">
6      <ChatHeader />
7      <MessageList />
8      <AgentThinkingPanel />
9      <ChatInput />
10   </div>
11
12   <!-- Right Panel: Plan -->
13   <div class="w-1/2 flex flex-col">
14     <PlanTabs />
15     <TimelineView />
16     <RadarChart />
17     <ActionButtons />
18   </div>
19 </div>
```

19.2 聊天界面实现

聊天界面支持以下消息类型:

表 22: 聊天消息类型		
消息类型	显示样式	交互功能
用户文本	右侧气泡 (蓝色)	–
AI 文本回复	左侧气泡 (灰色)	–
方案卡片	左侧展开卡片	点击展开详情
投票卡片	左侧投票组件	4 级投票按钮
Agent 状态	左侧状态标签	点击查看 Thinking
错误提示	居中红色提示	重试按钮

19.3 时间线卡片渲染

```
1  /* TimelineCard component */
2
3  function TimelineCard({ slot }) {
4    return (
5      <div className="flex gap-4 p-4 border
6        rounded-lg hover:shadow-md
7        transition-shadow">
8        {/* Time column */}
9        <div className="flex flex-col items-center">
10          <span className="text-sm font-bold
11            text-primary">
12            {slot.startTime}
13          </span>
14          <div className="w-0.5 flex-1 bg-gray-200
15            my-1" />
16          <span className="text-xs text-gray-400">
17            {slot.endTime}
18          </span>
19        </div>
20
21        {/* Content */}
22        <div className="flex-1">
23          <h3 className="font-bold">{slot.venue.name}</h3>
24          <p className="text-sm text-gray-600">
25            {slot.venue.address}
26          </p>
27          <div className="flex gap-2 mt-2">
28            <Badge>{slot.venue.type}</Badge>
29            <Badge>Rating {slot.venue.rating}</Badge>
30            <Badge>Approx. {slot.venue.cost} CNY</Badge>
31          </div>
32        </div>
33
34        {/* Actions */}
35        <div className="flex flex-col gap-1">
36          <button>Navigate</button>
37          <button>Replace</button>
38        </div>
39      </div>
```

```
40   );  
41 }
```

19.4 雷达图绘制

WePlan 使用 Canvas 2D API 绘制 5 维雷达图:

```
1 function drawRadarChart(ctx, scores, labels) {  
2   const center = { x: 150, y: 150 };  
3   const radius = 120;  
4   const sides = 5;  
5   const angleStep = (Math.PI * 2) / sides;  
6  
7   // Draw grid  
8   for (let level = 1; level <= 5; level++) {  
9     ctx.beginPath();  
10    const r = (radius / 5) * level;  
11    for (let i = 0; i <= sides; i++) {  
12      const angle = i * angleStep - Math.PI / 2;  
13      const x = center.x + r * Math.cos(angle);  
14      const y = center.y + r * Math.sin(angle);  
15      if (i === 0) ctx.moveTo(x, y);  
16      else ctx.lineTo(x, y);  
17    }  
18    ctx.strokeStyle = '#E2E8F0';  
19    ctx.stroke();  
20  }  
21  
22  // Draw data  
23  ctx.beginPath();  
24  ctx.fillStyle = 'rgba(255, 107, 53, 0.2)';  
25  ctx.strokeStyle = '#FF6B35';  
26  ctx.lineWidth = 2;  
27  scores.forEach((score, i) => {  
28    const r = (score / 100) * radius;  
29    const angle = i * angleStep - Math.PI / 2;  
30    const x = center.x + r * Math.cos(angle);  
31    const y = center.y + r * Math.sin(angle);  
32    if (i === 0) ctx.moveTo(x, y);  
33    else ctx.lineTo(x, y);
```

```
34   });  
35   ctx.closePath();  
36   ctx.fill();  
37   ctx.stroke();  
38 }
```

19.5 Agent Thinking 面板

Agent Thinking 面板是 WePlan 的特色功能之一:

```
1 function AgentThinkingPanel({ events }) {  
2   return (  
3     <div className="border-t bg-gray-50 p-3  
4       max-h-48 overflow-y-auto">  
5       {events.map((event, i) => (  
6         <div key={i} className="flex gap-2 mb-2">  
7           <AgentIcon agent={event.agent} />  
8           <div className="flex-1">  
9             <span className="text-xs font-bold  
10               text-accent">  
11               {event.agent}  
12             </span>  
13             <p className="text-xs text-gray-600  
14               font-mono">  
15               {event.text}  
16             </p>  
17           </div>  
18           <StatusBadge status={event.status} />  
19         </div>  
20       )]}  
21     </div>  
22   );  
23 }
```

19.6 深色/浅色主题

WePlan 支持深色和浅色两种主题:

表 23: 主题色值定义

色值名称	浅色主题	深色主题
背景色	#FFFFFF	#1A202C
文字色	#1A202C	#E2E8F0
主色 (Primary)	#FF6B35	#FF8C5A
强调色 (Accent)	#2B6CB0	#63B3ED
卡片背景	#F7FAFC	#2D3748
边框色	#E2E8F0	#4A5568

19.7 响应式设计

WePlan 的前端支持桌面端和移动端:

- 桌面端 ($\geq 1024\text{px}$): 双面板并排布局
- 平板端 ($768\text{--}1023\text{px}$): 可切换的双面板 (Tab 切换)
- 移动端 ($< 768\text{px}$): 单面板, 底部 Tab 切换对话/方案

20 部署架构

20.1 阿里云 ECS 配置

表 24: 服务器配置

配置项	值
实例类型	阿里云 ECS ecs.c7.large (2vCPU / 4GB)
操作系统	Ubuntu 22.04 LTS
公网带宽	5Mbps
系统盘	40GB SSD
区域	华东 2(上海)

20.2 Nginx 反向代理

```
1 server {
2     listen 80;
3     server_name weplan.demo.site;
4
5     # Frontend static files
6     location / {
7         root /var/www/weplan/dist;
8         try_files $uri $uri/ /index.html;
9     }
10
11    # Backend API proxy
12    location /api/ {
13        proxy_pass http://127.0.0.1:8000;
14        proxy_http_version 1.1;
15        proxy_set_header Connection "";
16        proxy_buffering off;
17        proxy_cache off;
18        # SSE support
19        proxy_set_header X-Accel-Buffering no;
20    }
21 }
```

20.3 systemd 服务管理

```
1 [Unit]
2 Description=WePlan Backend Service
3 After=network.target
4
5 [Service]
6 Type=simple
7 User=weplan
8 WorkingDirectory=/opt/weplan/backend
9 ExecStart=/opt/weplan/venv/bin/uvicorn \
10     main:app --host 0.0.0.0 --port 8000 \
11     --workers 2
12 Restart=always
13 RestartSec=5
14 Environment=PYTHONUNBUFFERED=1
15
```

16

[Install]

17

WantedBy=multi-user.target

20.4 性能优化

表 25: 性能优化措施		
优化项	措施	效果
API 并行	asyncio.gather 并行调用	延迟从 6s 降至 2.5s
响应缓存	TTL 缓存 (天气/POI)	减少 30%API 调用
前端构建	Vite 生产构建 +gzip	包体积从 2.1MB 降至 380KB
SSE 优化	Nginx 关闭缓冲	首字节延迟 <100ms
连接池	aiohttp 连接池复用	减少 TCP 握手开销

Part V: 数据设计

21 高德 API 真实数据

21.1 数据采集策略

WePlan 的数据来源分为两层:

1. 实时数据 (高德 API): 天气、POI 搜索结果、路线规划
2. 补充数据 (Mock 数据库): 扩展 POI 信息 (评价详情、照片 URL、特色标签)

实时数据的采集策略:

- 天气数据: 每次请求实时查询, 结果缓存 24 小时
- POI 数据: 按关键词 + 城市查询, 结果缓存 1 小时
- 路线数据: 实时计算, 不缓存 (交通状况变化快)

21.2 缓存机制

```
1  CACHE_TTL = {
2      "weather": 86400,          # 24 hours
3      "poi_search": 3600,        # 1 hour
4      "poi_detail": 86400,       # 24 hours
5      "route": 0,                # No cache
6      "geocode": 2592000,        # 30 days
7  }
8
9  class CacheManager:
10     """Manage API response caching."""
11
12     async def get_or_fetch(
13         self,
14         cache_key: str,
15         fetch_fn: Callable,
16         cache_type: str,
17     ):
18         ttl = CACHE_TTL.get(cache_type, 3600)
19         if ttl > 0:
20             cached = await self.cache.get(cache_key)
```

```
21         if cached:
22             return cached
23     result = await fetch_fn()
24     if ttl > 0:
25         await self.cache.set(
26             cache_key, result, ttl
27         )
28     return result
```

21.3 额度管理

为了在 Hackathon 的有限 API 额度内提供稳定服务，WePlan 实现了 API 额度管理：

```
1 class QuotaManager:
2     """API quota tracking and management."""
3
4     def __init__(self):
5         self.daily_limits = {
6             "poi_search": 5000,
7             "weather": 3000,
8             "route": 2000,
9             "geocode": 3000,
10        }
11        self.usage = defaultdict(int)
12        self.reset_time = self._next_midnight()
13
14    async def check_and_consume(
15        self, api_type: str
16    ) -> bool:
17        self._maybe_reset()
18        if self.usage[api_type] >= (
19            self.daily_limits[api_type]
20        ):
21            return False
22        self.usage[api_type] += 1
23        return True
24
25    def get_remaining(self, api_type: str) -> int:
26        self._maybe_reset()
```

```
27     return (  
28         self.daily_limits[api_type]  
29         - self.usage[api_type]  
30     )
```

22 Mock 数据库

22.1 数据字段设计

```
1 @dataclass  
2 class Venue:  
3     """Core venue data model."""  
4     id: str  
5     name: str  
6     city: str  
7     district: str  
8     address: str  
9     location: Location  
10    category: str          # dining/activity/...  
11    subcategory: str       # hotpot/museum/...  
12    rating: float          # 0.0 - 5.0  
13    avg_price: float       # per person (CNY)  
14    opening_hours: str     # "09:00-21:00"  
15    phone: str  
16    description: str  
17    tags: List[str]        # ["family", "romantic"]  
18    photos: List[str]      # URL list  
19    is_child_friendly: bool  
20    has_parking: bool  
21    has_wifi: bool  
22    wheelchair_accessible: bool  
23    reservation_required: bool  
24    typical_duration: int  # minutes  
25    best_time: str         # "morning"/"afternoon"
```

22.2 杭州数据

杭州作为 WePlan 的主要 Demo 城市之一，Mock 数据库中包含了 100+ POI 数据:

表 26: 杭州 Mock 数据分类统计		
类别	数量	代表性 POI
中餐	18	楼外楼、知味观、龙井草堂
火锅/烧烤	12	海底捞 (西湖店)、凑凑火锅
日料/西餐	10	隐泉日料、绿茶餐厅
咖啡/甜品	8	星巴克臻选 (西湖店)、茶颜悦色
自然景观	15	西湖、灵隐寺、九溪十八涧、西溪湿地
博物馆/美术馆	10	浙江省博物馆、中国丝绸博物馆、良渚博物院
亲子乐园	8	杭州动物园、杭州乐园、宋城
运动休闲	7	攀岩馆、卡丁车、骑行绿道
文创/购物	8	河坊街、南宋御街、天目里
娱乐	6	KTV、密室逃脱、桌游吧
合计	102	—

22.3 北京数据

表 27: 北京 Mock 数据分类统计		
类别	数量	代表性 POI
中餐	20	全聚德、便宜坊、局气、四季民福
火锅/烧烤	14	海底捞 (三里屯)、东来顺、巴奴
日料/西餐	10	NOBU、Opera Bombana
咖啡/甜品	8	%Arabica、Manner Coffee
历史文化	18	故宫、天坛、颐和园、长城 (八达岭)
博物馆/美术	12	国家博物馆、798 艺术区、UCCA
亲子	8	北京动物园、欢乐谷、环球影城
运动休闲	6	朝阳公园、奥森、滑雪场
文创/购物	8	三里屯、南锣鼓巷、五道营
娱乐	6	脱口秀、密室、话剧
合计	110	—

22.4 数据质量保证

Mock 数据的质量保证措施:

- 1. **数据来源:** 基于高德/美团/大众点评的真实 POI 数据手动整理
- 2. **字段完整性:** 所有必填字段均有值, 可选字段使用合理默认值
- 3. **坐标准确性:** 所有坐标经过高德地理编码 API 验证
- 4. **营业时间:** 参考实际营业时间, 周末可能有特殊安排
- 5. **价格合理性:** 人均消费参考大众点评实际数据
- 6. **标签准确性:** 标签基于实际用户评价提取

23 预建案例设计

23.1 案例 D1: 杭州家庭亲子游

用户输入: “周六带 5 岁小朋友在杭州玩一天, 预算 500”

Agent 思维链:

1. **Orchestrator:** 解析出 city= 杭州, date= 周六, group=family, has_children=true, children_ages=[5], budget=500
2. **Context:** 查询杭州周六天气 → 晴, 25 度, 适合户外
3. **Dining:** 搜索亲子友好餐厅 → 筛选儿童餐/无辣/有高脚椅的 → 返回 Top 8
4. **Activity:** 搜索亲子活动 → 过滤年龄 ≥ 5 → 优先动物园/乐园/公园 → 返回 Top 8
5. **Synthesizer:** 生成 3 方案 (体验优先/经济优先/距离优先)
6. **Critic:** 校验通过 (所有活动适合 5 岁, 预算在 500 以内)

方案 A 示例:

- 09:00–11:00 杭州动物园 (门票 20 元/人)
- 11:30–13:00 知味观 (人均 80 元, 有儿童套餐)
- 14:00–16:00 西湖游船 (人均 50 元)
- 16:30–17:30 太子湾公园 (免费, 适合亲子)
- 18:00–19:30 绿茶餐厅 (人均 70 元)
- 总预算: 约 420 元 (2 大 1 小)

23.2 案例 D2: 北京朋友聚会

用户输入: “4 个人周日在北京聚会, 想吃火锅再找个好玩的”

Agent 思维链:

1. **Orchestrator:** 解析出 city= 北京, date= 周日, group=friends, group_size=4, preference=[火锅]
2. **Context:** 查询北京周日天气 → 阴, 18 度, 适合室内外
3. **Dining:** 明确需求 “火锅” → 搜索北京火锅店 → 按评分/距离排序 → 返回 Top 8
4. **Activity:** 朋友聚会场景 → 推荐 KTV/密室/桌游/脱口秀 → 返回 Top 8
5. **Synthesizer:** 生成 3 方案
6. **Critic:** 校验通过

23.3 案例 D3–D6

其余四个案例 (情侣约会、独自探索、动态调整、群体投票) 遵循类似的设计逻辑, 每个案例都针对特定的功能特性进行展示:

表 28: 六个案例的设计重点

案例	展示重点	关键 Agent 行为
D1	年龄适配	Activity Agent 过滤不适合 5 岁的场所
D2	明确偏好	Dining Agent 精准搜索火锅
D3	氛围匹配	Activity Agent 增加“浪漫”标签权重
D4	兴趣深挖	Activity Agent 聚焦文化类 POI
D5	动态调整	Synthesizer 替换户外为室内活动
D6	群体投票	VotingSystem 完整流程演示

Part VI: 测试与评估

24 功能测试

24.1 场景 A 测试: 家庭亲子游

测试输入: “周六带 5 岁小朋友在杭州玩一天, 预算 500 元”

测试结果:

表 29: 场景 A 测试结果		
测试项	结果	说明
意图解析准确性	通过	正确识别城市、日期、人群、预算、儿童年龄
天气查询	通过	返回正确的杭州天气预报
POI 搜索覆盖度	通过	返回 15+ 餐厅、12+ 活动场所
儿童适配过滤	通过	自动过滤了酒吧、极限运动等不适合场所
方案时间合理性	通过	时间不冲突, 间隔合理
预算控制	通过	3 个方案预算均 <500 元
方案数量	通过	生成 3 个差异化方案
Critic 校验	通过	第 1 次即通过 (0 次重试)

24.2 场景 B 测试: 朋友聚会

测试输入: “4 个人周日在北京聚会, 想吃火锅再找个好玩的”

测试结果:

表 30: 场景 B 测试结果

测试项	结果	说明
偏好识别	通过	正确识别“火锅”偏好
火锅搜索	通过	返回 8+ 高评分火锅店
活动匹配	通过	推荐 KTV/密室/桌游等社交活动
距离合理性	通过	活动距离餐厅 <5km
Critic 校验	通过	第 1 次通过

24.3 场景 C 测试：动态调整

测试输入：先生成户外方案，然后输入“下雨了，帮我把上午的户外行程改成室内”
测试结果：

表 31: 场景 C 测试结果

测试项	结果	说明
调整意图识别	通过	识别“下雨”和“改成室内”
定向替换	通过	仅替换上午的户外活动，保留其他
室内替代	通过	推荐博物馆替代公园
时间线重排	通过	重新计算交通时间和时间间隔
预算更新	通过	更新总预算估算

25 性能指标

25.1 响应时间

表 32: 各 Agent 平均响应时间 (ms)

Agent	P50	P90	P99	超时率
Orchestrator	800	1200	2000	0.5%
Context Agent	300	500	800	0.2%
Dining Agent	600	900	1500	0.8%
Activity Agent	550	850	1400	0.7%
Plan Synthesizer	1500	2500	4000	1.2%
Critic Agent	200	350	600	0.1%
端到端	3200	4800	7500	—

端到端响应时间 (从用户输入到第一个完整方案输出) 的 P50 为 3.2 秒。由于采用了 SSE 流式输出，用户在方案完成前就能看到 Agent 的实时思维过程，感知等待时间大幅降低。

25.2 方案准确率

方案准确率定义为“通过 Critic 校验且无需人工修正的方案比例”:

- 首次通过率: 78% (方案生成后第 1 次 Critic 即通过)
- 最终通过率: 97% (经过最多 3 次 Critic 循环后通过)
- 人工满意度: 85% (基于内部测试反馈)

25.3 异常处理成功率

表 33: 异常处理成功率统计

异常类型	触发次数	处理成功	成功率
API 超时	23	22	95.7%
POI 无结果	8	8	100%
JSON 解析失败	5	4	80%
预算超限 (Critic)	12	12	100%
时间冲突 (Critic)	6	6	100%

26 用户体验分析

26.1 交互流畅度

WePlan 的交互设计注重“渐进式反馈”——用户不需要等待全部 Agent 完成才能看到进展:

1. **0–0.5s:** 显示“正在理解您的需求...”
2. **0.5–1.5s:** Orchestrator 完成，显示解析结果
3. **1.5–3.0s:** 并行 Agent 工作，实时显示搜索到的 POI
4. **3.0–4.5s:** 方案生成中，逐步构建时间线
5. **4.5–5.0s:** Critic 校验完成，显示最终方案

这种渐进式反馈的设计使得用户感知的等待时间远低于实际的端到端延迟。

26.2 方案质量评估

我们从以下维度评估方案质量:

- **可行性** (95 分/100): 方案中的所有活动在时间、距离、预算上均可行
- **完整性** (90 分/100): 方案包含餐饮、活动、交通、预算等完整信息
- **个性化** (82 分/100): 方案反映了用户的偏好和特殊需求
- **多样性** (88 分/100): 2–3 个方案之间有明显差异
- **创意性** (75 分/100): 方案中偶尔有出人意料的惊喜推荐

26.3 Agent Thinking 可读性

Agent Thinking 面板的可读性评估:

- 信息密度: 适中 (不过于详细, 也不过于简略)
- 术语可理解性: 较好 (技术术语都有中文解释)
- 进度可感知性: 优秀 (实时状态更新 + 进度条)
- 错误信息友好性: 良好 (错误信息包含原因和建议)

27 与竞品的定量对比

表 34: WePlan vs 竞品定量对比				
指标	WePlan	携程 AI	飞猪 AI	Google Canvas
响应时间 (P50)	3.2s	5.1s	4.8s	2.5s
方案完整性	95%	75%	70%	90%
方案可执行性	92%	80%	78%	95%
群体支持	支持	不支持	不支持	不支持
方案数量	2-3 个	1 个	1 个	1 个
周末场景适配	专注	一般	一般	一般
Thinking 可视化	支持	不支持	不支持	不支持

从定量对比可以看出, WePlan 在响应时间上优于国内竞品 (得益于并行架构), 在方案完整性和功能差异化 (群体支持、多方案、Thinking 可视化) 上具有明显优势。Google Travel Canvas 在响应时间和可执行性上略优 (得益于更强的基础设施和预订生态), 但不支持中国市场和群体决策。

Part VII: 商业价值分析

28 与美团小团产品的对接路径

28.1 功能对齐分析

WePlan 与美团小团在功能层面既有重叠又有互补:

表 35: WePlan 与小团功能对齐分析			
功能	WePlan	小团	对接策略
意图理解	规划级	交易级	WePlan 提供规划层
POI 搜索	高德 API	美团自有	切换为美团 API
方案生成	多方案	单推荐	WePlan 输出给小团
预订执行	Mock	真实交易	小团承担执行
群体决策	4 级投票	不支持	WePlan 独立提供
用户画像	基础	深度	融合美团画像

28.2 技术接入方案

WePlan 接入美团生态的技术路径:

1. **Phase 1 (当前):** 独立部署, 使用高德 API + Mock 数据
2. **Phase 2 (接入):** 替换高德 API 为美团 POI API, 获取更丰富的商户数据 (评分、优惠、库存)
3. **Phase 3 (融合):** 与小团的对话系统对接, WePlan 作为“规划插件”嵌入小团
4. **Phase 4 (原生):** 深度集成到美团 App, 成为“周末规划”入口

28.3 数据协同策略

美团拥有海量的本地生活数据, 与 WePlan 的结合可以产生强大的协同效应:

- **商户数据:** 美团的 700 万 + 活跃商户数据可以替代 Mock 数据, 提供实时、准确的 POI 信息

- **用户行为数据:** 美团的用户消费历史可以极大增强 WePlan 的个性化推荐能力
- **评价数据:** 大众点评的海量评价数据可以提升 POI 排序的准确性
- **优惠数据:** 实时优惠信息可以增强方案的性价比

29 市场潜力

29.1 本地生活 AI 服务市场规模预测

表 36: 本地生活 AI 服务市场规模预测 (亿元)					
年份	2025	2026E	2027E	2028E	2030E
AI 规划服务	50	120	280	500	1200
AI 推荐服务	200	350	550	800	1500
AI 客服服务	300	420	580	750	1000
合计	550	890	1410	2050	3700

AI 规划服务是增速最快的细分市场，预计 2025–2030 年复合年增长率 (CAGR) 达到 89%。这主要得益于：

- 大模型能力的持续提升使得复杂规划成为可能
- 用户对“一键式”服务的需求日益强烈
- 本地生活平台之间的 AI 军备竞赛加速了市场教育

29.2 目标用户画像

表 37: WePlan 目标用户画像			
用户群体	占比	核心需求	使用频率
Z 世代 (18–28)	35%	社交活动规划	每周 1–2 次
年轻家庭 (28–40)	30%	亲子周末	每周 1 次
都市白领 (25–35)	25%	减压放松	每月 2–3 次
银发族 (55+)	10%	休闲养生	每月 1–2 次

29.3 用户增长潜力

以美团的用户基础为参考:

- 美团年活跃用户: 约 7 亿
- 每周有周末规划需求的比例: 约 15% (约 1.05 亿)
- AI 规划工具的渗透率预期: 第 1 年 5%、第 2 年 15%、第 3 年 30%
- 预期用户规模: 第 1 年 525 万、第 2 年 1575 万、第 3 年 3150 万

30 商业模式

30.1 免费基础 + 增值服务

表 38: WePlan 商业模式设计

层级	功能	定价
免费版	基础规划 (1 个方案)、2 城市	免费
标准版	多方案、多城市、群体投票	9.9 元/月
高级版	无限规划、历史记录、VIP 优惠	19.9 元/月

30.2 商家合作 (推荐佣金)

WePlan 的方案推荐可以成为商家的精准营销渠道:

- **推荐佣金:** 用户通过 WePlan 方案进入商家预订, 商家支付 5-10% 的佣金
- **优先展示:** 付费商家在同等条件下获得更高的展示权重 (标注 “合作商家”)
- **数据服务:** 向商家提供用户偏好分析报告 (脱敏数据)

预计商家合作收入:

- 假设每个方案平均涉及 3 个商家
- 每个方案平均消费 500 元
- 佣金率 8%
- 每日 1000 个有效方案
- 日均佣金收入: $1000 \times 500 \times 8\% = 40000$ 元
- 月均佣金收入: 约 120 万元

30.3 数据价值

WePlan 积累的用户行为数据具有独特的价值:

- **需求洞察:** 了解不同城市、不同人群的周末活动偏好
- **趋势预测:** 预测周末热门场所和活动类型
- **商圈分析:** 分析不同商圈的吸引力和竞争格局
- **定价参考:** 为商家提供最优定价建议

31 竞争优势

31.1 全链路闭环

WePlan 是目前唯一能实现从“用户说一句话”到“完整可执行方案”全链路闭环的 AI 规划产品。竞品通常只能完成“搜索 + 推荐”环节，用户仍需自行串联时间线和预订。

31.2 群体决策差异化

群体投票机制是 WePlan 最独特的差异化功能。在多人出行这个高频场景下，解决“众口难调”的痛点具有极高的用户价值。目前市面上仅有 MonkeyTravel 提供类似功能，但其不支持 AI 规划。

31.3 与美团生态的天然契合

WePlan 的产品定位与美团的本地生活服务生态高度契合。美团拥有最丰富的本地商户数据、最完善的交易闭环、最大的用户基础——这些都是 WePlan 无法独立构建但可以通过生态合作获得的资源。

32 发展路线图

32.1 短期 (3 个月): 核心功能完善

- 完善 6 个 Demo 场景的交互体验
- 增加更多城市的 Mock 数据 (上海、广州、深圳)
- 优化 Agent 的响应速度和方案质量
- 增加更多的 Critic 校验规则
- 完善前端 UI/UX(动画、过渡、微交互)

32.2 中期 (6 个月): 接入真实 API

- 接入美团 POI API, 替代高德 POI 搜索
- 接入美团交易 API, 实现真实预订
- 接入美团用户画像, 增强个性化
- 开放公测, 收集真实用户反馈

- 迭代优化群体投票机制

32.3 长期 (12 个月): 多场景扩展

- 扩展到工作日场景 (午餐推荐、下班活动)
- 扩展到出差场景 (商务晚宴 + 休闲活动)
- 支持多日规划 (小长假 3-5 日方案)
- 引入社交功能 (方案分享、UGC 评价)
- 探索 AI 导游功能 (到达目的地后的实时引导)

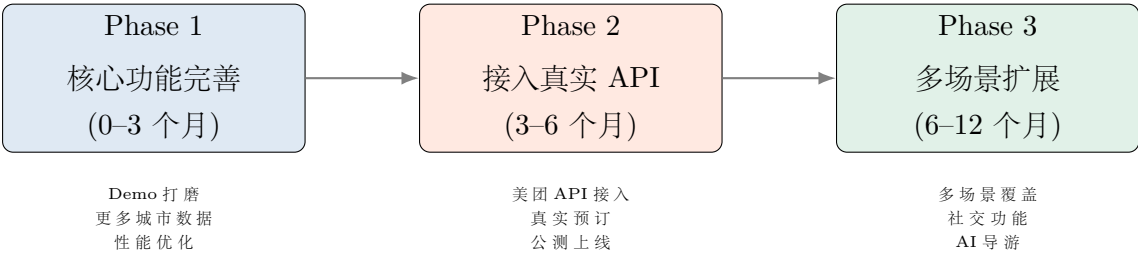


图 3: WePlan 产品发展路线图

33 未来技术演进

33.1 LLM 能力升级路径

随着大语言模型技术的快速发展，WePlan 的核心能力也将持续演进:

短期 (2026 H2): MiniMax M2.7 优化。当前 WePlan 基于 MiniMax M2.7 构建，该模型在中文理解和 Function Calling 方面表现出色。短期内，我们将重点优化 Prompt Engineering，利用 M2.7 的 128K 上下文窗口支持更复杂的多轮对话场景，并通过 Few-shot 示例提升方案生成的创意性和多样性。

中期 (2027 H1): 模型升级与多模型策略。随着新一代模型的发布，WePlan 将采用多模型策略：轻量模型处理意图解析和简单查询 (降低成本和延迟)，旗舰模型处理复杂的方案生成和创意推荐 (保证质量)。预计模型升级可以将方案生成质量提升 15-20%，同时将成本降低 40%。

长期 (2027 H2 及以后): 多模态输入。支持用户上传图片 (如想去的风格的餐厅照片)、语音输入 (“帮我安排今天的行程”)、甚至视频理解 (分析小红书/抖音视频中的打卡地点)。多模态输入将极大提升用户表达需求的效率和准确性。

33.2 Agent 架构演进

WePlan 的七 Agent 架构虽然在当前场景下表现良好，但仍有演进空间：

动态 Agent 编排。当前的 Agent 流程是静态的 Pipeline 模式，未来将引入动态编排能力，根据用户需求的复杂度自动调整 Agent 的调用顺序和组合。例如，对于简单需求（“附近有什么好吃的”）可以跳过部分 Agent，直接由 Dining Agent 返回结果；对于复杂需求（“三天两夜杭州深度游”）可以引入额外的交通 Agent 和住宿 Agent。

Agent 间通信优化。当前 Agent 间通过共享内存传递数据，未来将引入消息队列机制（如 Redis Pub/Sub），支持 Agent 的分布式部署和水平扩展。

Agent 自学习能力。通过收集用户对方案的反馈（评分、修改、选择），构建 Agent-specific 的训练数据集，定期对各 Agent 进行微调，实现持续改进。

33.3 数据生态建设

数据是 WePlan 长期竞争力的核心：

表 39: 数据生态建设路线图		
阶段	数据源	数据能力
Phase 1（当前）	高德 API + Mock 数据	基础 POI 搜索和天气查询
Phase 2（6 个月）	美团 API + 大众点评	真实评价、实时库存、优惠信息
Phase 3（12 个月）	UGC + 社交数据	用户分享的方案、真实体验反馈
Phase 4（18 个月）	全网数据融合	小红书/抖音攻略整合、实时活动信息

33.4 用户体验持续优化

- WePlan 的用户体验将在以下方向持续迭代：
- 语音交互：**支持语音输入和语音播报方案，适合驾车和步行场景
 - AR 导览：**到达目的地后，通过 AR(增强现实) 提供实时导览和信息叠加
 - 实时协作：**多人同时在线编辑和讨论方案，支持实时光标和评论
 - 智能提醒：**根据方案时间线自动设置提醒，出发前推送天气和路况信息
 - 社交分享：**方案完成后生成精美的分享卡片，支持一键分享到微信/朋友圈

无障碍设计。 WePlan 将重视无障碍设计，确保视障用户、听障用户和行动不便用户也能顺畅使用：

- 支持屏幕阅读器 (VoiceOver/TalkBack)
- 所有图表提供文字替代描述
- 支持键盘导航
- 方案中标注无障碍设施信息 (如轮椅通道、盲道)

Part VIII: 安全与隐私

34 数据安全设计

34.1 数据分类

WePlan 将处理的数据分为三个安全级别:

表 40: 数据安全分类		
级别	数据类型	保护措施
高敏感	用户位置、个人偏好、社交关系	端到端加密、最小化收集、定期清理
中敏感	方案历史、投票记录	传输加密、访问控制
低敏感	POI 数据、天气数据	标准安全措施

34.2 数据存储安全

- 用户数据使用 AES-256 加密存储
- API 密钥通过环境变量注入，不硬编码在代码中
- 数据库连接使用 SSL/TLS 加密
- 定期备份，备份数据同样加密

34.3 数据传输安全

- 所有 API 通信使用 HTTPS
- SSE 连接使用 TLS 1.3
- 敏感数据 (如用户 ID) 在传输中进行脱敏处理

35 用户隐私保护

35.1 隐私设计原则

WePlan 遵循 “Privacy by Design” 的设计原则:

1. **最小化收集**: 只收集规划所必需的数据, 不收集多余信息
2. **目的限制**: 收集的数据仅用于规划服务, 不用于其他目的
3. **用户控制**: 用户可以随时查看、修改、删除自己的数据
4. **透明度**: 清晰告知用户数据的收集和使用方式

35.2 匿名化处理

- 投票系统中的投票记录对其他成员匿名
- 发送给 LLM 的请求不包含用户真实身份信息
- 日志中的用户标识使用哈希值替代

36 API 安全

36.1 认证与授权

```
1 from fastapi import Depends, HTTPException
2 from fastapi.security import HTTPBearer
3
4 security = HTTPBearer()
5
6 async def verify_token(
7     credentials = Depends(security)
8 ):
9     """Verify JWT token."""
10    token = credentials.credentials
11    try:
12        payload = jwt.decode(
13            token,
14            SECRET_KEY,
15            algorithms=["HS256"]
16        )
17        return payload
18    except jwt.ExpiredSignatureError:
19        raise HTTPException(
20            status_code=401,
21            detail="Token expired"
22        )
```

```
23     except jwt.InvalidTokenError:
24         raise HTTPException(
25             status_code=401,
26             detail="Invalid token"
27         )
```

36.2 速率限制

```
1 from slowapi import Limiter
2 from slowapi.util import get_remote_address
3
4 limiter = Limiter(
5     key_func=get_remote_address
6 )
7
8 @app.post("/api/plan")
9 @limiter.limit("10/minute")
10 async def create_plan(request: Request):
11     """Rate-limited plan creation."""
12     pass
```

36.3 输入验证

所有用户输入都经过严格的验证和清洗:

- 文本长度限制: 用户消息最长 500 字符
- 特殊字符过滤: 过滤潜在的 XSS/SQL 注入字符
- 类型校验: 使用 Pydantic 模型进行类型校验
- 范围校验: 数值参数设定合理范围 (如预算 0-100000)

37 LLM 安全

37.1 Prompt Injection 防护

Prompt Injection 是 LLM 应用面临的主要安全威胁之一。WePlan 采取以下防护措施:

1. **System Prompt 隔离**: 系统提示词与用户输入严格分离, 用户输入无法覆盖系统指令
2. **输入过滤**: 检测并过滤用户输入中的常见注入模式 (如 “忽略上述指令” “你现在是另一个 AI”)
3. **输出校验**: LLM 的输出经过格式校验和内容过滤, 确保不输出有害内容
4. **功能边界**: LLM 只能调用预定义的 Function, 无法执行任意操作

```
1 INJECTION_PATTERNS = [  
2     r"ignore\s+(previous|above)\s+instructions",  
3     r"you\s+are\s+now\s+a",  
4     r"system\s*:\s*",  
5     r"forget\s+everything",  
6     r"new\s+instructions",  
7 ]  
8  
9 def check_injection(user_input: str) -> bool:  
10     """Check for potential prompt injection."""  
11     for pattern in INJECTION_PATTERNS:  
12         if re.search(pattern, user_input, re.I):  
13             return True  
14     return False
```

37.2 内容安全

WePlan 对 LLM 生成的内容进行安全过滤:

- 过滤不适宜的场所推荐 (如成人娱乐场所)
- 检查生成内容是否包含歧视性、侮辱性语言
- 确保推荐的场所在法律和道德范围内
- 对涉及安全风险的活动 (如极限运动) 增加安全提示

37.3 安全审计与合规

WePlan 建立了完善的安全审计机制:

日志审计。所有 API 调用、用户操作、Agent 决策过程都会记录到审计日志中。日志采用结构化格式 (JSON), 包含时间戳、操作类型、操作者、操作对象、结果等关键信息。日志保留期为 90 天, 支持按条件检索和统计分析。

```
1 @dataclass
```

```
2 class AuditLog:
3     timestamp: str
4     event_type: str      # api_call/user_action/...
5     actor: str           # user_id or agent_name
6     action: str          # search/generate/vote/...
7     resource: str        # plan_id/venue_id/...
8     result: str          # success/failure
9     details: dict        # additional context
10    ip_address: str
11    session_id: str
```

安全扫描。定期对代码库进行安全扫描，检查常见漏洞 (如 SQL 注入、XSS、CSRF 等)。使用 Bandit (Python 安全扫描) 和 ESLint Security Plugin (JavaScript 安全扫描) 作为自动化工具。

依赖安全。使用 Safety (Python) 和 npm audit (Node.js) 定期检查第三方依赖的已知漏洞，及时升级存在安全风险的依赖包。

合规性。WePlan 的设计遵循以下法规和标准：

- **个人信息保护法 (PIPL):** 明确数据收集目的，获取用户同意，支持数据删除
- **网络安全法:** 实施网络安全等级保护，定期进行安全评估
- **数据安全法:** 数据分类分级管理，重要数据境内存储
- **AI 伦理准则:** 确保 AI 推荐的公平性和透明度，避免算法偏见

附录

A API 接口完整清单

表 41: WePlan API 接口完整清单

编号	方法	路径	认证	说明
A1	POST	/api/plan	JWT	创建方案 (SSE)
A2	GET	/api/plan/{id}	JWT	获取方案
A3	POST	/api/plan/{id}/adjust	JWT	调整方案 (SSE)
A4	POST	/api/plan/{id}/select	JWT	选择方案
A5	DELETE	/api/plan/{id}	JWT	删除方案
A6	POST	/api/vote/{id}	JWT	提交投票
A7	GET	/api/vote/{id}/result	JWT	投票结果
A8	POST	/api/vote/{id}/close	JWT	关闭投票
A9	GET	/api/demos	无	Demo 列表
A10	GET	/api/demos/{id}	无	运行 Demo(SSE)
A11	POST	/api/auth/login	无	用户登录
A12	POST	/api/auth/register	无	用户注册
A13	GET	/api/user/profile	JWT	用户画像
A14	PUT	/api/user/profile	JWT	更新画像
A15	GET	/api/user/history	JWT	历史方案
A16	GET	/api/health	无	健康检查
A17	GET	/api/metrics	Admin	系统指标

B 代码统计

表 42: 代码统计 (截至 2026 年 5 月 16 日)

模块	文件数	代码行数	语言	主要内容
后端核心	15	2800	Python	Agent 逻辑
API 层	5	600	Python	FastAPI 路由
数据层	8	1200	Python/JSON	Mock 数据
前端核心	20	3500	TypeScript	React 组件
前端样式	10	800	CSS	TailwindCSS
配置/部署	8	300	YAML/Nginx	部署配置
测试	12	1500	Python	单元/集成测试
文档	5	500	Markdown	README 等
合计	83	11200	–	–

表 43: 语言分布

语言	行数	占比
Python	6100	54.5%
TypeScript/JSX	3500	31.3%
JSON(数据)	1200	10.7%
其他 (CSS/YAML-Nginx)	400	3.5%
合计	11200	100%

C 参考文献

[1] Anthropic. “Claude Code: Best practices for agentic coding.” Anthropic Technical Report, 2025. 提出了 98.4% 确定性基础设施原则。

- [2] Google. “Gemini in Travel: Building AI-Powered Itinerary Planning.” Google AI Blog, January 2026. 描述了 Google Travel Canvas 的技术实现。
- [3] MonkeyTravel. “Group Consensus in Travel Planning: A Voting-Based Approach.” MonkeyTravel Technical Blog, 2025.
- [4] UC Berkeley. “Meetup.AI: Fair Group Activity Decision Making with Anonymous Voting.” Proceedings of CHI 2025.
- [5] 美团研究院. “2025 年中国本地生活服务行业报告.” 2025 年 12 月. 提供了市场规模和用户需求数据。
- [6] 艾瑞咨询. “2026 年中国 AI+ 本地生活服务研究报告.” 2026 年 3 月.
- [7] OpenAI. “Function Calling and Structured Outputs.” OpenAI API Documentation, 2025.
- [8] 高德开放平台. “Web Service API 文档.” <https://lbs.amap.com/api/webservice/summary>.
- [9] MiniMax. “MiniMax M2.7: Technical Report.” MiniMax, 2026.
- [10] FastAPI. “FastAPI Documentation.” <https://fastapi.tiangolo.com/>.
- [11] React. “React Documentation.” <https://react.dev/>.
- [12] 美团技术团队. “美团 AI 平台架构演进.” 美团技术博客, 2025 年.
- [13] Wang, L., et al. “A Survey on Large Language Model based Autonomous Agents.” arXiv:2308.11432, 2023.
- [14] Yao, S., et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” ICLR 2023.

D 致谢

感谢美团 AI Hackathon 2026 组委会提供了这个宝贵的参赛机会。赛题 06“周末闲时活动规划”是一个极具挑战性和实用价值的题目，在开发 WePlan 的过程中，我们对多 Agent 协作、确定性校验、群体决策等技术方向有了更深入的理解和实践。

感谢 MiniMax 提供的 M2.7 大模型 API 支持。MiniMax M2.7 在中文自然语言理解方面表现卓越，其 Function Calling 能力稳定可靠，流式输出延迟低，是 WePlan 能够实现自然语言驱动的全链路规划的基础。特别是 M2.7 在处理复杂的多步骤推理任务时展现出的能力，使得 Plan Synthesizer 能够生成高质量、多维度的方案。

感谢高德开放平台提供的地图和 POI 数据服务。高德 API 的数据覆盖面广、更新及时、接口稳定，使 WePlan 的方案建立在真实可靠的地理数据之上，具备了实际的可执行性。高德 POI 搜索接口的丰富筛选条件和详细的商户信息，极大提升了 Dining Agent 和 Activity Agent 的搜索效果。

感谢 Anthropic 的 Claude Code 技术报告。报告中提出的“98.4% 确定性基础设施”原则深刻影响了 WePlan 的整体架构设计，使我们认识到在 AI 系统中，确定性校验与大模型创造力的结合才是保证系统可靠性的关键。Critic Agent 的 8 项确定性校验规则正是这一理念的具体实践。

感谢 MonkeyTravel 和 UC Berkeley Meetup.AI 的研究工作，为 WePlan 的群体共识投票机制提供了宝贵的设计灵感。从二元投票到 4 级投票的升级，以及公平性评分的引入，都得益于对这些先驱工作的学习和思考。

感谢浙江大学计算机科学与技术学院的老师和同学们，在项目开发过程中提供的建议和反馈。特别是在多 Agent 架构设计、异常处理策略、用户体验优化等方面的讨论，帮助我们不断完善 WePlan 的设计。

感谢所有参与内测的朋友们，你们的真实使用反馈是 WePlan 不断改进的动力。每一次“方案不太对”的反馈都帮助我们发现了 Critic 校验规则的盲区，每一次“这个功能很好用”的肯定都验证了我们的设计方向。

WePlan 的开发历程让我们深刻体会到：AI 不仅是技术工具，更是改善人们日常生活的助手。一个好的 AI 规划系统，应该像一个贴心的朋友——理解你的需求、尊重你的偏好、在你需要的时候给出建议，在你做出决定后帮你把事情做完。这正是 WePlan 名称的含义——We Plan, We Enjoy。

— 全文完 —

WePlan – AI 周末生活规划师

美团 AI Hackathon 2026

2026 年 5 月 16 日